

Express.js

80 Interview Questions & Answers

What is Express & why use it

Middleware deep dive (5 types)

Router & route organization

req.params, req.query, req.body

Body parsing & express.json()

All response methods

RESTful routing & status codes

Route parameter validation

Query string parsing

API versioning

Route wildcards & patterns

Route-level caching

express-validator

CORS configuration

helmet.js security headers

morgan request logging

compression middleware

Rate limiting with Redis

cookie-parser & cookies

express-session

JWT authentication

RBAC implementation

NoSQL injection prevention

File uploads with multer + S3

HTTP security headers

MVC pattern

Dependency injection

Repository pattern

WebSockets with Express

Server-Sent Events (SSE)

Multipart form + file validation

Graceful shutdown

Health check endpoints

Swagger/OpenAPI docs

PUT vs PATCH

Audit logging

→ All 80 questions answered with production code ←

→ Follow for Next: MongoDB (120 Q) → Auth & Security (60 Q) → DevOps (40 Questions) ←

■ Save • ■ Comment 'JS950' for full PDF • ■■ Repost

Kaushal Singh

Full-Stack Dev | MERN + React Native | Built Fintech Apps at Kanmotech Pvt. Ltd.

1M+ Impressions | Follow for Express • Node • MongoDB • Interview Prep

■ Section 4/10 | 80 Q&A | Next: MongoDB 120 Q | Comment 'JS950'

What is Express.js? Why use it over raw Node http?

■ Answer

Express is a minimal, fast, unopinionated web framework for Node.js built on the http module. It adds routing, middleware pipeline, request/response helpers, and error handling.

What raw Node.js requires (painful):

- Manually parse URL and route: `if(req.url === '/users' && req.method === 'GET')`
- Manually read body: `req.on('data', chunk => body += chunk)`
- Manually set headers: `res.writeHead(200, {'Content-Type': 'application/json'})`
- No middleware system — write everything from scratch

What Express provides:

- Clean routing: `app.get()`, `app.post()`, `app.put()`, `app.delete()`, `app.patch()`
- Middleware: `app.use()` — functions execute in sequence
- Body parsing: `express.json()`, `express.urlencoded()`
- Response helpers: `res.json()`, `res.send()`, `res.status()`, `res.redirect()`
- Router — modular route files
- Static file serving: `express.static()`

```

1 // Raw Node.js HTTP — every route is manual work
2 const http = require('http')
3 http.createServer((req, res) => {
4   if (req.url === '/users' && req.method === 'GET') {
5     let body = ''
6     req.on('data', chunk => body += chunk)
7     req.on('end', () => {
8       res.writeHead(200, { 'Content-Type': 'application/json' })
9       res.end(JSON.stringify({ users: [] }))
10    })
11  } else if (req.url === '/users' && req.method === 'POST') {
12    // repeat same body-reading boilerplate...
13  }
14 }).listen(3000)
15
16 // ■ Express — clean, readable, maintainable
17 const express = require('express')
18 const app = express()
19 app.use(express.json())
20
21 app.get('/users', async (req, res) => {
22   const users = await User.find()
23   res.json(users) // auto sets Content-Type: application/json
24 })
25
26 app.post('/users', async (req, res) => {
27   const user = await User.create(req.body) // body already parsed!
28   res.status(201).json(user)
29 })

```

```
31 app.listen(3000, () => console.log('Server on :3000'))
```

■ Interview Tip:

Express is the most downloaded npm package (50M+ weekly downloads). Despite being 14 years old, it's still the standard for Node.js REST APIs because of its simplicity, middleware ecosystem, and huge community.

What is middleware in Express? How does it work?

■ Answer

Middleware is a function with access to (req, res, next) that sits in the request-response pipeline. Each middleware can: read/modify req and res, end the request, or pass to next middleware.

Middleware execution order:

Request → MW1 → MW2 → MW3 → Route Handler → Response

5 types of middleware:

- Application-level — `app.use(fn)` — runs for all routes
- Router-level — `router.use(fn)` — runs for that router's routes
- Error-handling — 4 params (err, req, res, next) — MUST be last
- Built-in — `express.json()`, `express.urlencoded()`, `express.static()`
- Third-party — cors, morgan, helmet, multer, compression

Critical rules:

- Always call `next()` OR send a response — never leave request hanging
- Call `next(err)` to jump directly to error handler
- Error handler middleware MUST have exactly 4 parameters

```

1  const express = require('express')
2  const app = express()
3
4  // Middleware 1 — runs for ALL routes
5  app.use((req, res, next) => {
6    req.requestId = require('crypto').randomUUID()
7    req.startTime = Date.now()
8    console.log(`[${req.requestId}] ${req.method} ${req.url}`)
9    next() // MUST call next or request hangs!
10 })
11
12 // Middleware 2 — body parser
13 app.use(express.json({ limit: '10mb' }))
14
15 // Middleware 3 — auth (route-specific)
16 const auth = async (req, res, next) => {
17   const token = req.headers.authorization?.split(' ')[1]
18   if (!token) return res.status(401).json({ error: 'No token' })
19   try {
20     req.user = jwt.verify(token, process.env.JWT_SECRET)
21     next() // authenticated — pass to route handler
22   } catch {
23     next(new Error('Invalid token')) // pass to ERROR handler
24   }
25 }
26
27 // Route with middleware chain
28 app.get('/dashboard', auth, (req, res) => {

```

```

29   res.json({ user: req.user }) // auth ran first

```

```
30  })
31
32  // Response timing middleware
33  app.use((req, res, next) => {
34    res.on('finish', () => {
35      console.log(`${req.method} ${req.url} ${res.statusCode} ${Date.now()-req.startTime}ms`)
36    })
37    next()
38  })
39
40  // Error handler – LAST middleware, 4 params required!
41  app.use((err, req, res, next) => {
42    res.status(err.statusCode || 500).json({ error: err.message })
43  })
```

■ Interview Tip:

The most common Express bug: forgetting to call `next()` in a middleware that's supposed to pass control. The request hangs until timeout (default 120 seconds on most servers). Always either call `next()` or send a response.

Q3

What is the Express Router? How to organize routes?

■ Answer

Express Router creates mini-applications with their own middleware and routes. It enables modular route organization — each resource gets its own router file.

Why use Router:

- Separation of concerns — routes by resource
- Group-level middleware — apply auth to all routes in a router
- Cleaner app.js — just mounts routers
- Testable in isolation

Recommended project structure:

routes/auth.js — POST /login, /register, /refresh, /logout
 routes/users.js — GET/POST /users, GET/PUT/DELETE /users/:id
 routes/products.js — full CRUD for products
 routes/index.js — combines all routers
 app.js — mounts combined router at /api/v1

```

1 // routes/users.js
2 const router = require('express').Router()
3 const { authenticate, authorize } = require('../middleware/auth')
4 const { validate } = require('../middleware/validate')
5 const { createUserSchema, updateUserSchema } = require('../schemas/user')
6 const ctrl = require('../controllers/users')
7
8 // Apply auth to ALL routes in this router
9 router.use(authenticate)
10
11 // GET /api/v1/users — list all (admin only)
12 router.get('/', authorize('admin'), ctrl.list)
13
14 // GET /api/v1/users/me — current user profile
15 router.get('/me', ctrl.getProfile)
16
17 // GET /api/v1/users/:id — get specific user
18 router.get('/:id', ctrl.getById)
19
20 // POST /api/v1/users — create user
21 router.post('/', authorize('admin'), validate(createUserSchema), ctrl.create)
22
23 // PUT /api/v1/users/:id — update user
24 router.put('/:id', validate(updateUserSchema), ctrl.update)
25
26 // DELETE /api/v1/users/:id — delete user (admin only)
27 router.delete('/:id', authorize('admin'), ctrl.delete)
28
29 // Nested route: GET /api/v1/users/:id/orders
30 router.get('/:id/orders', ctrl.getOrders)
31
32 module.exports = router
  
```

```
33
34 // routes/index.js – barrel file
35 const router = require('express').Router()
36 router.use('/auth', require('./auth'))
37 router.use('/users', require('./users'))
38 router.use('/products', require('./products'))
39 router.use('/orders', require('./orders'))
40 module.exports = router
41
42 // app.js – clean and simple
43 app.use('/api/v1', require('./routes'))
```

■ Interview Tip:

The router-per-resource pattern keeps files small (50-100 lines each). When a bug is reported on a user endpoint, you immediately know to look in routes/users.js, controllers/users.js, and services/users.js. No searching through a 2000-line app.js.

What is req.params, req.query, req.body?

■ Answer

These three properties give access to the three main ways clients send data to Express.

req.params — URL path parameters:

- Defined in route with :paramName
- GET /users/123 → req.params.id = '123' (always a string!)
- Used for resource identifiers

req.query — URL query string:

- Everything after ? in the URL
- GET /products?page=2&limit=20 → req.query = { page:'2', limit:'20' }
- Values are always strings — convert with parseInt(), parseFloat()
- Used for filtering, pagination, sorting

req.body — request body:

- Data sent in POST/PUT/PATCH body
- Requires middleware to parse: express.json() or express.urlencoded()
- undefined if no body parser middleware
- Used for creating/updating resources

```
1 // Route: /api/products/:categoryId/items/:itemId
2 app.get('/api/products/:categoryId/items/:itemId', (req, res) => {
3   console.log(req.params)
4   // { categoryId: 'electronics', itemId: '456' }
5
6   // ALWAYS convert params to correct types!
7   const itemId = req.params.itemId // string '456'
8   const numId = parseInt(itemId, 10) // number 456
9 })
10
11 // GET /api/products?page=2&limit=20&sort=price&order=asc&category=phones
12 app.get('/api/products', (req, res) => {
13   console.log(req.query)
14   // { page:'2', limit:'20', sort:'price', order:'asc', category:'phones' }
15
16   const page = parseInt(req.query.page) || 1
17   const limit = Math.min(parseInt(req.query.limit) || 20, 100) // max 100
18   const { sort = 'createdAt', order = 'desc', category } = req.query
19
20   // Use validated/converted values
21   const skip = (page - 1) * limit
22 })
23
24 // POST /api/users with JSON body
25 app.post('/api/users', express.json(), async (req, res) => {
26   console.log(req.body)
27   // { name: 'Kaushal', email: 'k@example.com', age: 26 }
```

```
29 // NEVER trust raw body – always validate first!
30 const { name, email } = req.body // basic destructure
31 // Better: validate with Zod/Joi first
32 })
33
34 // Form data
35 app.post('/api/contact', express.urlencoded({ extended: true }), (req, res) => {
36   const { name, message } = req.body // from HTML form POST
37 })
```

■ Interview Tip:

req.params values are ALWAYS strings — even if the param looks like a number. Always convert: `const id = parseInt(req.params.id)` or use Zod's `z.coerce.number()`. This is a frequent source of bugs when comparing MongoDB ObjectIds.

What is express.json() and body parsing?

■ Answer

express.json() is built-in middleware that parses incoming requests with JSON payloads. Without it, req.body is undefined for JSON POST/PUT requests.

Body parsing middlewares:

- express.json() — parses application/json
- express.urlencoded() — parses application/x-www-form-urlencoded (HTML forms)
- express.raw() — parses as Buffer (for webhooks, binary data)
- express.text() — parses as plain text
- multer — parses multipart/form-data (file uploads)

Important options:

- limit — max body size (default '100kb') — ALWAYS set this!
- strict — only accept arrays and objects (default true)
- type — custom content-type to parse

Why limit matters:

- Without limit, attacker can send 1GB JSON body
- Server loads all of it into memory — crash!

```

1  const express = require('express')
2  const app = express()
3
4  // Parse JSON bodies — application/json
5  app.use(express.json({
6    limit: '10mb',    // max body size (protect against large payloads)
7    strict: true,     // only accept objects/arrays (default)
8    verify: (req, res, buf, encoding) => {
9      // Store raw body for webhook signature verification
10     req.rawBody = buf
11   })
12 })))
13
14 // Parse URL-encoded form data — application/x-www-form-urlencoded
15 app.use(express.urlencoded({
16   extended: true,    // use qs library (supports nested objects)
17   limit: '10mb'
18 })))
19
20 // Static files
21 app.use('/public', express.static('public', {
22   maxAge: '1d',      // browser cache for 1 day
23   etag: true,        // enable ETags for conditional requests
24   index: false       // don't serve index.html automatically
25 })))
26
27 // Raw body for Stripe webhooks (need raw body for HMAC verification!)
```

```

28 app.post('/webhooks/stripe',
```

```

29   express.raw({ type: 'application/json' })), // raw Buffer, not parsed JSON
30   async (req, res) => {
31     const signature = req.headers['stripe-signature']
32     const event = stripe.webhooks.constructEvent(
33       req.body,    // raw Buffer
34       signature,
35       process.env.STRIPE_WEBHOOK_SECRET
36     )
37     await handleStripeEvent(event)
38     res.json({ received: true })
39   }
40 )
41
42 // ■ Common mistake: express.json() AFTER routes
43 // Routes that come BEFORE app.use(express.json()) will have req.body = undefined!
44 // Always put body parsers BEFORE routes.

```

■ Interview Tip:

Always use `express.raw()` for webhook routes (Stripe, GitHub, Razorpay). These platforms include a HMAC signature to verify the request is legitimate. If you parse JSON first, the raw body Buffer is lost and signature verification fails.

Q6 What are the response methods in Express?

Answer

Express adds many helper methods to Node's `ServerResponse` for sending different response types.

Most used response methods:

- `res.json(obj)` — send JSON (sets `Content-Type: application/json`)
- `res.send(data)` — send string/Buffer/object (smart content-type)
- `res.status(code)` — set HTTP status code (chainable)
- `res.redirect(url)` — redirect to another URL
- `res.sendFile(path)` — send file with proper headers
- `res.download(path, filename)` — send file as download
- `res.render(view, data)` — render template (EJS, Handlebars)
- `res.set(header, value)` — set response header
- `res.cookie(name, val, opts)` — set cookie
- `res.clearCookie(name)` — remove cookie
- `res.end()` — end without sending body

Chaining:

- `res.status(201).json(data)`
- `res.status(404).send('Not found')`

```
1 // Common response patterns
2 app.get('/users/:id', async (req, res) => {
3   const user = await User.findById(req.params.id)
4
5   // 404 - not found
6   if (!user) return res.status(404).json({ error: 'User not found' })
7
8   // 200 - success with data
9   res.json(user) // shorthand for res.status(200).json(user)
10 })
11
12 app.post('/users', async (req, res) => {
13   const user = await User.create(req.body)
14   res.status(201).json({ user, message: 'User created' }) // 201 Created
15 })
16
17 // Redirect
18 app.get('/old-path', (req, res) => {
19   res.redirect(301, '/new-path') // 301 permanent, 302 temporary
20 })
21
22 // Set multiple headers
23 res.set({
24   'X-Request-Id': req.requestId,
25   'X-Response-Time': `${Date.now() - req.startTime}ms`,
26   'Cache-Control': 'no-store'
27 })
```

```
28
```

```

29 // Download file
30 app.get('/export', async (req, res) => {
31   const filePath = await generateExport()
32   res.download(filePath, 'report.csv', (err) => {
33     if (err) console.error('Download error:', err)
34     fs.unlink(filePath, () => {}) // clean up temp file
35   })
36 })
37
38 // Send file
39 app.get('/avatar/:userId', async (req, res) => {
40   const path = `./uploads/${req.params.userId}.jpg`
41   res.sendFile(path, { root: __dirname }, (err) => {
42     if (err) res.status(404).json({ error: 'Avatar not found' })
43   })
44 })
45
46 // Cookie responses
47 res.cookie('refreshToken', token, {
48   httpOnly: true, secure: true, sameSite: 'strict',
49   maxAge: 7 * 24 * 60 * 60 * 1000 // 7 days in ms
50 })
51 res.clearCookie('refreshToken') // logout

```

■ Interview Tip:

Never call two response methods! After `res.json()` or `res.send()`, the response headers are sent. Calling `res.json()` again causes 'Error: Cannot set headers after they are sent to the client'. Use early returns to prevent this.

Q7 What is the request object (req) in Express?

■ Answer

Express extends Node's IncomingMessage with many useful properties for accessing request data.

Key req properties:

- req.method — 'GET', 'POST', 'PUT', 'DELETE', 'PATCH'
- req.url — full URL including query string
- req.path — pathname only (no query string)
- req.params — URL route parameters
- req.query — parsed query string object
- req.body — parsed request body (needs middleware)
- req.headers — all request headers
- req.cookies — parsed cookies (needs cookie-parser)
- req.ip — client IP (respects X-Forwarded-For)
- req.protocol — 'http' or 'https'
- req.secure — true if HTTPS
- req.hostname — hostname from Host header
- req.get(header) — get specific header value
- req.is(type) — check Content-Type

```
1 app.use((req, res, next) => {
2   // Method and URL
3   console.log(req.method)    // 'GET'
4   console.log(req.url)       // '/users/123?include=posts'
5   console.log(req.path)      // '/users/123'
6
7   // Headers
8   console.log(req.headers)   // { authorization: 'Bearer...', ... }
9   console.log(req.get('Authorization')) // 'Bearer ...'
10  console.log(req.get('Content-Type'))  // 'application/json'
11
12  // Client info
13  console.log(req.ip)         // '203.0.113.42' or ':::1' for localhost
14  console.log(req.hostname)   // 'api.example.com'
15  console.log(req.protocol)   // 'https'
16  console.log(req.secure)     // true/false
17
18  // Content type check
19  if (req.is('application/json')) {
20    // body is JSON
21  }
22  if (req.is('multipart/form-data')) {
23    // file upload
24  }
25
26  // Custom properties (set by previous middleware)
27  console.log(req.user)       // set by auth middleware
28  console.log(req.requestId)  // set by request ID middleware
```

```
29 console.log(req.startTime) // set by timing middleware
```

```
30
31   next()
32 })
33
34 // req.ip with reverse proxy (Nginx in front)
35 // Configure Express to trust proxy:
36 app.set('trust proxy', 1) // trust first proxy (Nginx)
37 // Now req.ip returns real client IP from X-Forwarded-For
38 // Without this, req.ip = Nginx server IP (not useful!)
```

■ **Interview Tip:**

If your Express app is behind a load balancer or reverse proxy (Nginx), set `app.set('trust proxy', 1)`. Otherwise `req.ip` returns the proxy's IP, not the real client's IP — and rate limiting by IP won't work correctly.

Q8 What is app.set() and app.get() for settings?

■ Answer

Express has a settings system to configure application-level behaviors.

Key Express settings:

- 'trust proxy' — trust X-Forwarded-* headers from proxies
- 'x-powered-by' — show 'Express' in X-Powered-By header (set false in production!)
- 'env' — environment ('development' or 'production')
- 'views' — template views directory
- 'view engine' — default template engine
- 'case sensitive routing' — /Foo and /foo treated differently
- 'strict routing' — /foo and /foo/ treated differently
- 'query parser' — 'simple', 'extended', or custom function
- 'json spaces' — JSON indentation in development

app.set(name, val) — set setting

app.get(name) — get setting value

app.enable(name) — set to true

app.disable(name) — set to false

```
1  const express = require('express')
2  const app = express()
3
4  // Security: hide that you're using Express
5  app.disable('x-powered-by') // removes 'X-Powered-By: Express' header
6  // Or use helmet() which also does this
7
8  // Trust proxy (REQUIRED when behind Nginx/load balancer!)
9  app.set('trust proxy', 1) // trust first hop
10 // app.set('trust proxy', 'loopback, 10.0.0.0/8') // trust specific IPs
11
12 // Template engine setup
13 app.set('views', path.join(__dirname, 'views'))
14 app.set('view engine', 'ejs')
15
16 // Pretty-print JSON in development
17 if (process.env.NODE_ENV === 'development') {
18   app.set('json spaces', 2) // res.json() will be indented
19 }
20
21 // Case sensitive routing
22 app.set('case sensitive routing', true) // /Users !== /users
23
24 // Strict routing
25 app.set('strict routing', true) // /users !== /users/
26
27 // Custom query parser (using qs for nested objects)
```

```
28 const qs = require('qs')
```

```
29 app.set('query parser', str => qs.parse(str, {
30     allowDots: true,
31     depth: 5
32 })))
33 // Now: ?filter[age][gt]=18 parses to { filter: { age: { gt: '18' } } }
34
35 // Get a setting value
36 console.log(app.get('env'))           // 'development'
37 console.log(app.get('trust proxy'))   // 1
38 console.log(app.enabled('x-powered-by')) // false after disable()
```

■ **Interview Tip:**

Always disable('x-powered-by') in production. Exposing your framework version helps attackers target known vulnerabilities. Also use helmet() which sets many security headers including hiding the X-Powered-By header.

What is app.locals and res.locals?

■ Answer

app.locals and res.locals store data accessible across the application or per-request.

app.locals:

- Data available for the LIFETIME of the application
- Accessible in template views as local variables
- Use for: app config, settings, constants
- Same object for all requests

res.locals:

- Data scoped to a SINGLE request-response cycle
- Set in middleware, available in later middleware and views
- Cleared after each request
- Use for: authenticated user, request-specific data

Both are available in views (EJS, Handlebars):

- {{ appName }} — from app.locals
- {{ currentUser }} — from res.locals

```

1  const express = require('express')
2  const app = express()
3
4  // app.locals - application-wide data
5  app.locals.appName = 'MyApp'
6  app.locals.version = process.env.npm_package_version
7  app.locals.supportEmail = 'support@myapp.com'
8  app.locals.startTime = new Date()
9
10 // In EJS template: <title><%= appName %></title>
11
12 // res.locals - per-request data
13 // Middleware sets user for all routes
14 app.use(async (req, res, next) => {
15   const token = req.cookies?.token
16   if (token) {
17     try {
18       const decoded = jwt.verify(token, process.env.JWT_SECRET)
19       res.locals.user = await User.findById(decoded.userId)
20       res.locals.isAuthenticated = true
21     } catch { res.locals.isAuthenticated = false }
22   }
23   next()
24 })
25
26 // Add standard response metadata
27 app.use((req, res, next) => {
28   res.locals.requestId = req.requestId

```

```

29   res.locals.timestamp = new Date().toISOString()

```

```

30   next()
31 })
32
33 // Route uses res.locals data
34 app.get('/dashboard', (req, res) => {
35   if (!res.locals.isAuthenticated) return res.redirect('/login')
36   res.render('dashboard') // template accesses user, isAuthenticated automatically
37 })
38
39 // Override res.json to include standard fields
40 app.use((req, res, next) => {
41   const originalJson = res.json.bind(res)
42   res.json = (data) => {
43     return originalJson({ data, requestId: res.locals.requestId })
44   }
45   next()
46 })

```

■ Interview Tip:

res.locals is perfect for the authenticated user pattern. Set res.locals.user in auth middleware once, and every route handler can access it without re-querying the database or passing it through function parameters.

Q10 Explain the Express request-response lifecycle

■ Answer

Understanding the full lifecycle helps debug issues and design better middleware.

Complete lifecycle:

1. TCP connection established (or reused)
2. HTTP request received (headers + body stream)
3. Express parses URL, method, headers
4. Middleware stack executes (in order defined)
5. Router matches URL to route handler
6. Route-specific middleware runs
7. Route handler generates response
8. Response sent to client
9. Cleanup (res.on('finish') callbacks fire)

Error path:

If any middleware calls `next(err)`, execution jumps directly to the first error handler middleware (4-param function)

Key events:

- req events: 'data', 'end', 'error', 'close'
- res events: 'finish', 'close', 'error'

```
1  const express = require('express')
2  const app = express()
3
4  // Step 1: Global middleware (runs for every request)
5  app.use((req, res, next) => {
6    req.startTime = Date.now()
7    next()
8  })
9
10 // Step 2: Body parsing
11 app.use(express.json())
12
13 // Step 3: Auth middleware
14 app.use(authenticate) // sets req.user if valid token
15
16 // Step 4: Router matches URL
17 app.use('/api/orders', ordersRouter) // matches /api/orders/*
18
19 // Inside ordersRouter:
20 // Step 5: Router-level middleware
21 ordersRouter.use((req, res, next) => {
22   if (!req.user) return res.status(401).json({ error: 'Not authenticated' })
23   next()
24 })
25
26 // Step 6: Route handler
```

```
27 ordersRouter.get('/:id', async (req, res, next) => {
```

```

28   try {
29     const order = await Order.findById(req.params.id)
30     if (!order) return res.status(404).json({ error: 'Not found' })
31     res.json(order) // Step 7: response sent
32   } catch(err) {
33     next(err) // jumps to error handler!
34   }
35 })
36
37 // Step 8: Cleanup (always fires after response)
38 app.use((req, res, next) => {
39   res.on('finish', () => {
40     const duration = Date.now() - req.startTime
41     logger.info({ method: req.method, url: req.url, status: res.statusCode, duration })
42   })
43   next()
44 })
45
46 // Error handler – catches errors from any middleware/route
47 app.use((err, req, res, next) => {
48   res.status(err.statusCode || 500).json({ error: err.message })
49 })

```

■ Interview Tip:

The `res.on('finish')` event fires after the response is sent — perfect for logging. Never log in the route handler because you don't yet know the final status code. Log in 'finish' where you have access to both `req` and the final `res.statusCode`.

Q11

What is RESTful routing? HTTP methods?

■ Answer

REST (Representational State Transfer) defines standard conventions for API endpoints. HTTP methods map to CRUD operations on resources.

REST conventions:

GET /resources — list all (200)
 POST /resources — create new (201)
 GET /resources/:id — get one (200)
 PUT /resources/:id — replace (full update) (200)
 PATCH /resources/:id — partial update (200)
 DELETE /resources/:id — delete (200 or 204)

Nested resources:

GET /users/:id/posts — posts by user
 POST /users/:id/posts — create post for user
 GET /users/:id/posts/:postId — specific post

HTTP status codes to know:

- 200 OK, 201 Created, 204 No Content
- 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
- 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests
- 500 Internal Server Error, 503 Service Unavailable

```

1  const router = require('express').Router()
2
3  // Collection endpoints
4  router.get('/products', ctrl.list)           // 200 + array
5  router.post('/products', ctrl.create)       // 201 + created resource
6
7  // Resource endpoints
8  router.get('/products/:id', ctrl.getById)   // 200 + resource
9  router.put('/products/:id', ctrl.replace)   // 200 + full updated resource
10 router.patch('/products/:id', ctrl.update)  // 200 + partial updated
11 router.delete('/products/:id', ctrl.delete) // 204 No Content
12
13 // Status code patterns:
14
15 // Successful create
16 res.status(201).json({ product: newProduct, message: 'Created' })
17
18 // Successful delete (no body)
19 res.status(204).send()
20
21 // Resource not found
22 res.status(404).json({ error: 'Product not found', id: req.params.id })
23
24 // Validation error

```

```

25 res.status(400).json({

```

```

26   error: 'Validation failed',
27   details: [{ field: 'email', message: 'Invalid email format' }]
28 })
29
30 // Duplicate resource
31 res.status(409).json({ error: 'Email already exists' })
32
33 // Unauthorized (not logged in)
34 res.status(401).json({ error: 'Authentication required' })
35
36 // Forbidden (logged in but no permission)
37 res.status(403).json({ error: 'Admin access required' })
38
39 // Rate limited
40 res.status(429).json({
41   error: 'Too many requests',
42   retryAfter: 60 // seconds
43 })

```

■ Interview Tip:

Use 401 when user is NOT authenticated (no token). Use 403 when user IS authenticated but lacks permission. This is the most commonly confused HTTP status code pair in interviews — get it right.

What is route parameter validation?

■ Answer

Route parameters should always be validated before using them in database queries. Invalid params cause MongoDB errors or security issues if not checked.

Why validate route params:

- MongoDB throws `CastError` if invalid `ObjectId`
- Could expose database errors to client
- SQL injection in relational DBs
- Unexpected behavior with malformed input

Validation approaches:

- Manual check with regex
- Zod schema for full validation
- `express-validator` middleware
- `param()` method on router

router.param() hook:

- Runs before route handlers for a named param
- Perfect for loading resources from DB
- Attached to the router, not individual routes

```
1 const router = require('express').Router()
2 const { z } = require('zod')
3 const mongoose = require('mongoose')
4
5 // Method 1: router.param() – runs for every :id in this router
6 router.param('id', async (req, res, next, id) => {
7   // Validate MongoDB ObjectId format
8   if (!mongoose.Types.ObjectId.isValid(id)) {
9     return res.status(400).json({ error: 'Invalid ID format' })
10  }
11
12  // Pre-load the resource
13  const product = await Product.findById(id)
14  if (!product) {
15    return res.status(404).json({ error: 'Product not found' })
16  }
17
18  req.product = product // attach to request
19  next()
20 })
21
22 // Now these routes don't need to fetch the product themselves!
23 router.get('/products/:id', (req, res) => {
24   res.json(req.product) // already loaded by param hook
25 })
26
```

```
27 router.patch('/products/:id', async (req, res) => {
```

```

28   const updated = await req.product.save() // already loaded!
29   res.json(updated)
30 })
31
32 // Method 2: Zod validation middleware
33 const mongoIdSchema = z.object({
34   params: z.object({ id: z.string().regex(/^[0-9a-fA-F]{24}$/, 'Invalid MongoDB ID') })
35 })
36
37 router.get('/orders/:id', validate(mongoIdSchema), async (req, res) => {
38   const order = await Order.findById(req.params.id)
39   if (!order) return res.status(404).json({ error: 'Order not found' })
40   res.json(order)
41 })
42
43 // Method 3: Inline check (simplest)
44 router.delete('/users/:id', async (req, res) => {
45   if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
46     return res.status(400).json({ error: 'Invalid user ID' })
47   }
48   await User.findByIdAndDelete(req.params.id)
49   res.status(204).send()
50 })

```

■ Interview Tip:

router.param() is a DRY pattern for resource loading. Instead of calling User.findById in every single route that has :userId, define it once in router.param('userId', ...) and every route gets req.user automatically.

What is query string parsing and validation?

■ Answer

Query strings carry optional parameters for filtering, sorting, and pagination. Always validate and sanitize query params before using in DB queries.

Common query string patterns:

- Pagination: ?page=1&limit=20
- Sorting: ?sortBy=price&order=asc
- Filtering: ?status=active&category=electronics
- Search: ?q=laptop&fields=name,price
- Date range: ?startDate=2024-01-01&endDate=2024-12-31

Security risks without validation:

- MongoDB operator injection: ?status[\$ne]=inactive
- XSS via reflected values
- Denial of service with huge limit values

Best practice:

- Validate with Zod/Joi
- Whitelist allowed sort fields
- Set max limit
- Coerce string types to numbers

```

1  const { z } = require('zod')
2
3  // Define query schema with Zod
4  const listProductsQuery = z.object({
5    page: z.coerce.number().int().positive().default(1),
6    limit: z.coerce.number().int().positive().max(100).default(20),
7    sortBy: z.enum(['price', 'name', 'createdAt', 'rating']).default('createdAt'),
8    order: z.enum(['asc', 'desc']).default('desc'),
9    category: z.string().max(50).optional(),
10   minPrice: z.coerce.number().nonnegative().optional(),
11   maxPrice: z.coerce.number().positive().optional(),
12   search: z.string().max(100).optional(),
13   inStock: z.coerce.boolean().optional()
14 })
15
16 router.get('/products', async (req, res, next) => {
17   try {
18     // Validate and coerce query params
19     const query = listProductsQuery.parse(req.query)
20     // After parse: page=number, limit=number, sortBy=string
21
22     // Build MongoDB query safely
23     const filter = {}
24     if (query.category) filter.category = query.category
25     if (query.search) filter.$text = { $search: query.search }
26
27     if (query.inStock !== undefined) filter.stock = query.inStock ? { $gt: 0 } : 0
  
```

```

27   if (query.minPrice || query.maxPrice) {
28     filter.price = {}
29     if (query.minPrice) filter.price.$gte = query.minPrice
30     if (query.maxPrice) filter.price.$lte = query.maxPrice
31   }
32
33   const skip = (query.page - 1) * query.limit
34   const [data, total] = await Promise.all([
35     Product.find(filter).sort({ [query.sortBy]: query.order === 'asc' ? 1 : -1 })
36       .skip(skip).limit(query.limit).lean(),
37     Product.countDocuments(filter)
38   ])
39
40   res.json({
41     data, pagination: {
42       page: query.page, limit: query.limit, total,
43       totalPages: Math.ceil(total / query.limit),
44       hasNext: query.page * query.limit < total
45     }
46   })
47 } catch(err) { next(err) }
48 })

```

■ Interview Tip:

Never use req.query values directly in MongoDB queries without sanitization. `?filter[$where]=malicious` creates a MongoDB operator injection. Use `express-mongo-sanitize` middleware AND `Zod` schema validation for double protection.

Q14 How to implement API versioning in Express?

■ Answer

API versioning allows breaking changes without disrupting existing clients.

Versioning strategies:

- URL path versioning — /api/v1/ vs /api/v2/ — MOST COMMON, explicit
- Header versioning — Accept: application/vnd.api.v2+json
- Query param — /api/users?version=2

URL versioning (recommended):

- Most visible and easiest to understand
- Easy to test in browser/curl
- Cacheable (URL is the key)

Migration approach:

- Never delete v1 immediately
- Deprecate with headers (warn clients)
- Set sunset date
- Keep old version running 6-12 months

```

1  const express = require('express')
2  const app = express()
3
4  // Version 1 router
5  const v1Router = express.Router()
6  v1Router.get('/users', require('./v1/controllers/users').list)
7  v1Router.post('/users', require('./v1/controllers/users').create)
8  v1Router.get('/products', require('./v1/controllers/products').list)
9
10 // Version 2 router (breaking changes: new fields, different format)
11 const v2Router = express.Router()
12 v2Router.get('/users', require('./v2/controllers/users').list)
13 v2Router.post('/users', require('./v2/controllers/users').create)
14 v2Router.get('/users/:id/profile', require('./v2/controllers/users').profile) // new!
15
16 // Deprecation middleware for v1
17 const deprecationNotice = (req, res, next) => {
18   res.setHeader('Deprecation', 'true')
19   res.setHeader('Sunset', 'Sat, 01 Jan 2026 00:00:00 GMT')
20   res.setHeader('Link', '</api/v2>; rel="successor-version"')
21   res.setHeader('Warning', '299 - "Deprecated API version"')
22   next() // still works, just warns
23 }
24
25 // Mount versioned routers
26 app.use('/api/v1', deprecationNotice, v1Router)
27 app.use('/api/v2', v2Router)
28

```

```

29 // Versioning via Accept header (content negotiation)

```

```
30 app.get('/api/users', (req, res) => {
31   const version = req.accepts(['application/vnd.api.v2+json', 'application/json'])
32   if (version === 'application/vnd.api.v2+json') {
33     return res.json(usersV2Format())
34   }
35   res.json(usersV1Format())
36 })
37
38 // Shared middleware across versions
39 app.use(['/api/v1', '/api/v2'], sharedRateLimiter, sharedAuthMiddleware)
```

■ Interview Tip:

Start versioning from day one, even if you only have v1. Adding versioning later requires changing all existing client URLs. The Deprecation and Sunset response headers are standard RFC 8594 — API clients can parse these to warn developers automatically.

Q15 What is route-level vs app-level middleware?

■ Answer

Middleware can be applied at different scopes — globally or to specific routes/routers.

Application-level (`app.use`):

- Runs for ALL requests to the application
- Applied in order of definition
- Use for: logging, body parsing, security headers, CORS

Router-level (`router.use`):

- Runs only for routes in that router
- Apply to groups of routes (e.g., all admin routes)
- Use for: auth for specific sections, scoped rate limits

Route-level (`inline in route`):

- Runs only for that specific route
- Array of middleware: [auth, validate, handler]
- Cleanest for route-specific logic

Order of execution:

`app.use` → `router.use` → route middleware → route handler

```
1  const app = require('express')()
2  const apiRouter = require('express').Router()
3  const adminRouter = require('express').Router()
4
5  // ■■ APP LEVEL – runs for every request ■■
6  app.use(require('morgan')('combined')) // log all requests
7  app.use(require('helmet')())           // security headers
8  app.use(require('cors')({ origin: process.env.CLIENT_URL }))
9  app.use(express.json({ limit: '10mb' }))
10
11 // ■■ ROUTER LEVEL – runs for /api/* routes ■■
12 apiRouter.use(rateLimit({ windowMs: 15*60*1000, max: 100 }))
13 apiRouter.use(authenticate) // all API routes require auth
14
15 // ■■ NESTED ROUTER LEVEL – runs for /api/admin/* ■■
16 adminRouter.use(authorize('admin')) // all admin routes require admin role
17 adminRouter.use(auditLog)           // log all admin actions
18
19 adminRouter.get('/users', getUsersController)
20 adminRouter.delete('/users/:id', deleteUserController)
21
22 // ■■ ROUTE LEVEL – runs only for this specific route ■■
23 apiRouter.post('/products',
24   authorize('admin', 'editor'), // only admins/editors
25   validate(createProductSchema), // validate this route's input
26   uploadImages,                 // handle file upload
27   createProductController        // actual handler
```

```
28 )
29
30 // Mounting
31 apiRouter.use('/admin', adminRouter)
32 app.use('/api/v1', apiRouter)
33
34 // 404 handler – catches unmatched routes
35 app.use((req, res) => res.status(404).json({ error: `Route ${req.path} not found` }))
36
37 // Error handler – catches errors from all routes
38 app.use((err, req, res, next) => {
39   res.status(err.statusCode || 500).json({ error: err.message })
40 })
```

■ Interview Tip:

Middleware order is critical. Putting `authenticate()` as `app.use()` means ALL routes require auth — including `/health` and `/login`. Use router-level middleware instead to apply auth only to protected route groups.

Q16

What is error-handling middleware? The 4-parameter signature?

■ Answer

Error handling middleware catches errors thrown or passed via `next(err)` from any route.

The 4-parameter signature is required:

- (err, req, res, next) — Express identifies error middleware by this signature
- Without all 4 params, Express treats it as regular middleware
- Must be defined LAST, after all routes

Error types to handle:

- Operational errors — expected: `NotFound`, `Unauthorized`, `Validation`
- Programmer errors — bugs: `TypeError`, `ReferenceError`

Error categorization:

- `err.statusCode` — HTTP status to return
- `err.isOperational` — `true` = expected error, safe to expose
- Operational: return `err.message` to client
- Programmer: return generic message, log full error

```

1 // Custom error classes
2 class AppError extends Error {
3   constructor(message, statusCode = 500, code = 'INTERNAL_ERROR') {
4     super(message)
5     this.statusCode = statusCode
6     this.code = code
7     this.isOperational = true
8     Error.captureStackTrace(this, this.constructor)
9   }
10 }
11
12 class NotFoundError extends AppError {
13   constructor(resource = 'Resource') { super(`${resource} not found`, 404, 'NOT_FOUND') }
14 }
15 class ValidationError extends AppError {
16   constructor(msg, details) {
17     super(msg, 400, 'VALIDATION_ERROR')
18     this.details = details
19   }
20 }
21 class UnauthorizedError extends AppError { constructor(m='Unauthorized') { super(m, 401) } }
22 class ForbiddenError extends AppError { constructor() { super('Forbidden', 403) } }
23 class ConflictError extends AppError { constructor(m) { super(m, 409) } }
24
25 // asyncHandler – wrap async routes to catch errors
26 const asyncHandler = fn => (req, res, next) =>
27   Promise.resolve(fn(req, res, next)).catch(next)
28
29 // Route – no try/catch needed with asyncHandler

```

```

30 router.get('/users/:id', asyncHandler(async (req, res) => {

```

```

31  const user = await User.findById(req.params.id)
32  if (!user) throw new NotFoundError('User') // caught by asyncHandler!
33  res.json(user)
34  )))
35
36  // GLOBAL ERROR HANDLER — must be LAST, must have 4 params
37  app.use((err, req, res, next) => {
38    const isDev = process.env.NODE_ENV === 'development'
39
40    // Log all errors
41    if (err.statusCode >= 500 || !err.isOperational) {
42      console.error('ERROR:', { url: req.url, error: err.message, stack: err.stack })
43    }
44
45    // Handle Mongoose validation errors
46    if (err.name === 'ValidationError') {
47      return res.status(400).json({
48        error: 'Validation failed',
49        details: Object.values(err.errors).map(e => ({ field: e.path, message: e.message }))
50      })
51    }
52
53    // Handle Mongoose duplicate key
54    if (err.code === 11000) {
55      const field = Object.keys(err.keyValue)[0]
56      return res.status(409).json({ error: `${field} already exists` })
57    }
58
59    res.status(err.statusCode || 500).json({
60      error: err.isOperational ? err.message : 'Internal Server Error',
61      code: err.code,
62      ...(isDev && { stack: err.stack }),
63      ...(err.details && { details: err.details })
64    })
65  })

```

■ Interview Tip:

The `asyncHandler` wrapper is essential for Express 4 and below. Without it, async errors are NOT caught by the global error handler — they become unhandled promise rejections. Express 5 (currently in beta) handles async errors natively.

Q17 What is the difference between `app.use()` and `app.get()`?

■ Answer

`app.use()` and `app.get()` both add handlers to Express's middleware stack, but with key differences.

`app.use(path?, fn):`

- Matches ANY HTTP method (GET, POST, PUT, DELETE, etc.)
- Path matching: `/api` matches `/api`, `/api/users`, `/api/anything`
- Path is a PREFIX match (not exact)
- Used for: middleware, mounting sub-routers

`app.get(path, fn):`

- Matches ONLY GET method
- Path is EXACT match (unless pattern used)
- Used for: route handlers

Other route methods:

- `app.post()` — POST only
- `app.put()` — PUT only
- `app.patch()` — PATCH only
- `app.delete()` — DELETE only
- `app.all()` — all methods, exact path
- `app.route()` — chain multiple methods

```

1  const app = require('express')()
2
3  // app.use() — any method, prefix match
4  app.use('/api', (req, res, next) => {
5    console.log('This runs for ALL methods to /api/*')
6    // Runs for: GET /api, POST /api/users, DELETE /api/users/1
7    next()
8  })
9
10 // app.get() — GET only, exact match
11 app.get('/api/users', (req, res) => {
12   // Only runs for: GET /api/users
13   // Does NOT run for: POST /api/users or GET /api/users/123
14   res.json(users)
15 })
16
17 // app.all() — all methods, exact match
18 app.all('/api/users/:id', (req, res, next) => {
19   console.log('Runs for any method to /api/users/:id exactly')
20   next()
21 })
22
23 // app.route() — chain multiple methods on same path
24 app.route('/api/products')
25   .get(async (req, res) => { res.json(await Product.find()) })

```

```

26   .post(auth, validate(schema), async (req, res) => {

```

```

27     const p = await Product.create(req.body)
28     res.status(201).json(p)
29   })
30
31   app.route('/api/products/:id')
32     .get(async (req, res) => { /* get one */ })
33     .put(auth, async (req, res) => { /* update */ })
34     .delete(auth, admin, async (req, res) => { /* delete */ })
35
36   // Route patterns
37   app.get('/files/:filename(\\.(jpg|png|gif|$))', serveImage) // regex in param
38   app.get('/user*', handler) // wildcard - matches /user, /users, /user-profile
39   app.get('/user(s)?', handler) // optional character

```

■ Interview Tip:

app.use('/api', router) mounts the router at /api and strips the /api prefix for internal routing. So if you mount at /api and your router has route /users, the full URL is /api/users. This is called path stripping.

Q18 What are route wildcards and regex patterns?

■ Answer

Express supports wildcards and regex patterns in route paths for flexible matching.

Path patterns:

- Exact: `/users` — matches only `/users`
- Named params: `/users/:id` — captures `/users/123`
- Optional: `/users/:id?` — matches `/users` and `/users/123`
- Wildcard: `/files/*` — matches any path starting with `/files/`
- Optional char: `/colou?r` — matches `/color` and `/colour`
- One or more: `/ab+cd` — matches `/abcd`, `/abbcd`, `/abbbcd`
- Zero or more: `/ab*cd` — matches `/abcd`, `/ab123cd`
- Groups: `/a(bc)?d` — matches `/ad` and `/abcd`

Named param constraints:

- `:id(\d+)` — numeric only
- `:slug([a-z-]+)` — lowercase letters and hyphens

Regex routes:

- `app.get(/^\/api\/v\d\/, handler)` — version regex

```

1  const app = require('express')()
2
3  // Named param with regex constraint
4  app.get('/users/:id(\d+)', (req, res) => {
5    // Only matches numeric IDs: /users/123
6    // /users/abc returns 404 automatically
7    res.json({ id: parseInt(req.params.id) })
8  })
9
10 app.get('/posts/:slug([a-z0-9-]+)', (req, res) => {
11   // Matches: /posts/my-first-post
12   // /posts/My Post with spaces → 404
13   res.json({ slug: req.params.slug })
14 })
15
16 // Optional parameter
17 app.get('/products/:category?', (req, res) => {
18   const { category } = req.params
19   if (category) return res.json({ products: filteredByCategory(category) })
20   res.json({ products: allProducts })
21 })
22
23 // Wildcard — serve SPA fallback
24 app.get('*', (req, res) => {
25   // SPA catch-all — let React Router handle navigation
26   res.sendFile(path.join(__dirname, 'public', 'index.html'))
27 })

```

```
29 // Regex route
30 app.get(/^\/(about|contact|help)$/, (req, res) => {
31   const page = req.path.slice(1) // 'about', 'contact', or 'help'
32   res.sendFile(`./pages/${page}.html`)
33 })
34
35 // Route ordering matters – more specific BEFORE wildcard
36 app.get('/products/featured', ctrl.featured) // must come first!
37 app.get('/products/:id', ctrl.getById)      // would match 'featured' too!
```

■ Interview Tip:

Route ordering is critical. Express matches routes in the order they're defined. If `/products/:id` comes before `/products/featured`, then 'featured' gets matched as an id. Always put specific routes before parameterized routes.

How to implement route-level caching?

■ Answer

Route-level caching reduces database queries for frequently accessed, slow-changing data.

Caching patterns for Express routes:

- Cache middleware — check Redis before hitting DB
- Response cache — cache entire HTTP response
- ETag caching — client-side conditional requests
- Cache-Control headers — tell CDN/browser how to cache

What to cache at route level:

- Product lists — change rarely, accessed frequently
- Category/tag lists — very static
- Public user profiles
- Aggregated statistics

What NOT to cache:

- Authenticated user data (or use user-scoped keys)
- Cart, checkout, payments
- Real-time data
- POST/PUT/DELETE responses

```
1  const redis = require('ioredis').createClient()
2
3  // Cache middleware factory
4  const cacheRoute = (ttlSeconds = 300, keyFn = null) => {
5    return async (req, res, next) => {
6      // Build cache key
7      const key = keyFn ? keyFn(req) : `route:${req.originalUrl}`
8
9      try {
10       const cached = await redis.get(key)
11       if (cached) {
12         res.setHeader('X-Cache', 'HIT')
13         res.setHeader('X-Cache-Key', key)
14         return res.json(JSON.parse(cached))
15       }
16     } catch (err) {
17       console.warn('Cache read error:', err.message) // don't fail on cache error
18     }
19
20     // Intercept res.json to cache the response
21     const originalJson = res.json.bind(res)
22     res.json = async (data) => {
23       try {
24         await redis.setex(key, ttlSeconds, JSON.stringify(data))
25         res.setHeader('X-Cache', 'MISS')
26       } catch (err) {
```

```
27       console.warn('Cache write error:', err.message)
```

```

28     }
29     return originalJson(data)
30   }
31   next()
32 }
33 }
34
35 // Public routes – cache by URL
36 router.get('/products', cacheRoute(600), ctrl.list) // 10 min
37 router.get('/categories', cacheRoute(3600), ctrl.getCategories) // 1 hour
38 router.get('/products/:id', cacheRoute(300, req => `product:${req.params.id}`), ctrl.get)
39
40 // Invalidate cache on mutation
41 router.post('/products', authenticate, authorize('admin'), async (req, res, next) => {
42   const product = await Product.create(req.body)
43
44   // Clear related caches
45   const pattern = 'route:/products*'
46   const keys = await redis.keys(pattern)
47   if (keys.length) await redis.del(...keys)
48   await redis.del(`category:${product.category}`) // clear category cache too
49
50   res.status(201).json(product)
51 })
52
53 // ETag for client-side caching
54 router.get('/products/:id', async (req, res) => {
55   const product = await Product.findById(req.params.id)
56   const etag = require('crypto').createHash('md5').update(product.updatedAt.toISOString()).digest('hex')
57   if (req.headers['if-none-match'] === etag) return res.status(304).end()
58   res.setHeader('ETag', etag)
59   res.setHeader('Cache-Control', 'public, max-age=300')
60   res.json(product)
61 })

```

■ Interview Tip:

Cache invalidation is the hardest part of caching. Use patterns: cache keys like 'route:/products?page=1' plus 'product:123', and when a product is updated, delete both. The redis KEYS command is for development only — use redis SCAN in production for pattern matching.

What is the express-validator library?

■ Answer

express-validator is middleware for validating and sanitizing Express request data. It wraps the validator.js library with Express middleware integration.

Core functions:

- `body('field')` — validate `req.body` field
- `param('field')` — validate `req.params` field
- `query('field')` — validate `req.query` field
- `header('name')` — validate request header
- `validationResult(req)` — check for validation errors

Common validators:

- `.isEmail()` — valid email format
- `.isLength({min, max})` — string length
- `.isNumeric()` — numeric value
- `.isMongoid()` — valid MongoDB ObjectId
- `.notEmpty()` — not empty string
- `.trim()` — sanitize whitespace
- `.escape()` — sanitize HTML characters

Zod vs express-validator:

- Zod: better TypeScript integration, cleaner syntax
- express-validator: older, more tutorials, Express-native

```
1  const { body, param, query, validationResult } = require('express-validator')
2
3  // Reusable error handler
4  const handleValidationErrors = (req, res, next) => {
5    const errors = validationResult(req)
6    if (!errors.isEmpty()) {
7      return res.status(400).json({
8        error: 'Validation failed',
9        details: errors.array().map(e => ({ field: e.path, message: e.msg }))
10     })
11   }
12   next()
13 }
14
15 // Validation chain for user registration
16 const registerValidation = [
17   body('name')
18     .trim().notEmpty().withMessage('Name is required')
19     .isLength({ min: 2, max: 50 }).withMessage('Name must be 2-50 chars'),
20
21   body('email')
22     .trim().normalizeEmail().isEmail().withMessage('Invalid email'),
```

```

24 body('password')
25   .isLength({ min: 8 }).withMessage('Min 8 characters')
26   .matches(/[A-Z]/).withMessage('Need uppercase letter')
27   .matches(/[0-9]/).withMessage('Need a number'),
28
29 body('age')
30   .optional()
31   .isInt({ min: 18, max: 120 }).withMessage('Must be 18-120')
32   .toInt(), // sanitize: convert to integer
33
34   handleValidationErrors // check errors after all validators
35 ]
36
37 router.post('/users', registerValidation, ctrl.create)
38
39 // Param validation
40 router.get('/users/:id', [
41   param('id').isMongoId().withMessage('Invalid user ID'),
42   handleValidationErrors
43 ], ctrl.getById)
44
45 // Query validation
46 router.get('/products', [
47   query('page').optional().isInt({ min: 1 }).toInt(),
48   query('limit').optional().isInt({ min: 1, max: 100 }).toInt(),
49   query('sortBy').optional().isIn(['price', 'name', 'createdAt']),
50   handleValidationErrors
51 ], ctrl.list)

```

■ Interview Tip:

Regardless of whether you use Zod or express-validator, always validate input. The `handleValidationErrors` middleware pattern is clean — put all validators in an array and the error handler at the end. Any validation failure returns 400 with details.

Q21 What is CORS? How to configure it in Express?

■ Answer

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks cross-origin requests by default. Your API must explicitly allow them.

What CORS controls:

- Which origins can access your API
- Which HTTP methods are allowed
- Which request headers are allowed
- Whether credentials (cookies) can be sent

Preflight requests:

- Browser sends OPTIONS request first for POST/PUT/PATCH/DELETE
- Checks if the actual request is permitted
- Server must respond to OPTIONS with CORS headers

The cors package:

- Handles CORS headers and preflight automatically
- Dynamic origin allows different configs per request

```
1  const cors = require('cors')
2
3  // ■ Too permissive – don't use in production
4  app.use(cors()) // allows ALL origins
5
6  // ■ Production config – whitelist specific origins
7  const allowedOrigins = new Set([
8    'https://app.mysite.com',
9    'https://admin.mysite.com',
10   process.env.NODE_ENV === 'development' && 'http://localhost:3000',
11   process.env.NODE_ENV === 'development' && 'http://localhost:5173' // Vite
12  ].filter(Boolean))
13
14  app.use(cors({
15    origin: (origin, callback) => {
16      // Allow requests with no origin (mobile apps, Postman, curl)
17      if (!origin || allowedOrigins.has(origin)) return callback(null, true)
18      callback(new Error(`CORS blocked: ${origin} not allowed`))
19    },
20    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
21    allowedHeaders: ['Content-Type', 'Authorization', 'X-Request-Id'],
22    credentials: true, // allow cookies to be sent
23    maxAge: 86400, // preflight cache: 24 hours (reduces OPTIONS calls)
24    optionsSuccessStatus: 200 // some old browsers choke on 204
25  }))
26
27  // Handle preflight for ALL routes
28  app.options('*', cors()) // respond to OPTIONS preflight
```

```
30 // Route-specific CORS (e.g., public API endpoint)
31 const publicCors = cors({ origin: '*', methods: ['GET'] })
32 app.get('/api/public/prices', publicCors, pricesController)
33
34 // ■ Common mistake: CORS + credentials = any origin is INSECURE
35 // cors({ origin: '*', credentials: true }) // NEVER DO THIS!
36 // Must use specific origins when credentials: true
```

■ **Interview Tip:**

credentials: true with origin: '*' is a security misconfiguration — browsers won't allow it anyway and will show a CORS error. When you need cookies/auth headers across origins, you **MUST** specify the exact allowed origins.

What is helmet.js? What headers does it set?

Answer

Helmet.js sets 15 HTTP response headers to protect against common web vulnerabilities. One line of code that provides significant security improvements.

Headers helmet sets by default:

- Content-Security-Policy — prevents XSS by whitelisting script sources
- X-Content-Type-Options: nosniff — prevent MIME sniffing
- X-Frame-Options: SAMEORIGIN — prevent clickjacking (in iframes)
- X-XSS-Protection: 0 — disabled (CSP handles this now)
- Strict-Transport-Security — force HTTPS for 1 year
- X-Download-Options — no IE downloads from your site
- Referrer-Policy — control referrer header
- Permissions-Policy — disable camera, mic, etc.

Headers removed by helmet:

- X-Powered-By: Express — hides framework version

CSP is the most important:

- Blocks inline scripts (prevent XSS)
- Whitelists allowed script sources

```
1 const helmet = require('helmet')
2
3 // Simple - use all defaults
4 app.use(helmet())
5
6 // Customized for your specific app
7 app.use(helmet({
8   // Content Security Policy
9   contentSecurityPolicy: {
10     directives: {
11       defaultSrc: ['self'],
12       scriptSrc: [
13         'self',
14         'cdn.jsdelivr.net', // allowed CDN
15         'nonce-RANDOMNONCE' // for inline scripts
16       ],
17       styleSrc: ['self', 'unsafe-inline', 'fonts.googleapis.com'],
18       imgSrc: ['self', 'data:', '*.cloudinary.com', '*.amazonaws.com'],
19       connectSrc: ['self', 'api.stripe.com'],
20       fontSrc: ['self', 'fonts.gstatic.com'],
21       objectSrc: ['none'], // no plugins
22       upgradeInsecureRequests: [], // upgrade http to https
23     },
24   },
25   // HSTS - tell browsers to only use HTTPS
26   hsts: {
```

```
27     maxAge: 31536000, // 1 year
```

```

28     includeSubDomains: true,
29     preload: true           // for HSTS preload list
30   },
31   // Disable old headers not needed
32   xPoweredBy: false, // explicit (helmet already removes it)
33 )))
34
35 // Generate CSP nonce per-request for inline scripts
36 app.use((req, res, next) => {
37   res.locals.cspNonce = require('crypto').randomBytes(16).toString('base64')
38   next()
39 })
40
41 // What helmet headers look like in response:
42 // Content-Security-Policy: default-src 'self'; script-src 'self' cdn.jsdelivr.net
43 // X-Content-Type-Options: nosniff
44 // X-Frame-Options: SAMEORIGIN
45 // Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
46 // Referrer-Policy: no-referrer

```

■ Interview Tip:

Always put helmet() as the FIRST middleware (before routes, before cors, before everything). This ensures security headers are set on every response including error responses. Many security audits fail because security headers are missing on error pages.

Q23 What is morgan? Request logging in Express?

■ Answer

Morgan is an HTTP request logger middleware for Express. It logs incoming requests with configurable format.

Built-in format tokens:

- 'combined' — Apache combined format (production standard)
- 'common' — Apache common format
- 'dev' — colorized development format
- 'short' — shorter than default
- 'tiny' — minimal output

Custom tokens:

- `morgan.token('name', fn)` — define custom field
- Use `:name` in format string

Production logging best practice:

- Use JSON format for log aggregation (ELK, Datadog)
- Include: `requestId`, `userId`, `duration`, `status`
- Skip health check endpoints
- Don't log sensitive data (auth headers, passwords)

```
1  const morgan = require('morgan')
2  const logger = require('./logger') // pino or winston
3
4  // Development — colorized, human-readable
5  if (process.env.NODE_ENV === 'development') {
6    app.use(morgan('dev'))
7    // Output: POST /api/users 201 14.123 ms - 284
8  }
9
10 // Production — JSON format for log aggregation
11 if (process.env.NODE_ENV === 'production') {
12   // Custom token: request ID
13   morgan.token('request-id', (req) => req.requestId)
14   morgan.token('user-id', (req) => req.user?.id || 'anonymous')
15   morgan.token('body-size', (req) => req.headers['content-length'] || 0)
16
17   app.use(morgan((tokens, req, res) => {
18     return JSON.stringify({
19       type: 'http',
20       requestId: tokens['request-id'](req, res),
21       userId: tokens['user-id'](req, res),
22       method: tokens.method(req, res),
23       url: tokens.url(req, res),
24       status: parseInt(tokens.status(req, res)),
25       duration: parseFloat(tokens['response-time'](req, res)),
26       size: parseInt(tokens.res(req, res, 'content-length')) || 0,
```

```
27   ip: tokens['remote-addr'](req, res),
```

```
28     userAgent: tokens['user-agent'](req, res)
29   })
30 }, {
31   // Skip health checks (too noisy in logs)
32   skip: (req) => req.url === '/health' || req.url === '/ready'
33 })
34 }
35
36 // Stream to custom logger (pino/winston)
37 app.use(morgan('combined', {
38   stream: {
39     write: (message) => logger.info(message.trim())
40   }
41 })))
```

■ Interview Tip:

Skip health check endpoints in your HTTP logs. Kubernetes probes /health every 10 seconds — that's 6 log entries per minute just from health checks. Skip them in Morgan and keep your logs focused on real traffic.

What is compression middleware?

■ Answer

Compression middleware gzips/deflates HTTP responses to reduce payload size. Reduces bandwidth and improves load times — typically 70-80% size reduction for JSON.

When compression helps most:

- JSON API responses — excellent compression
- HTML, CSS, JavaScript — excellent
- Already compressed formats (JPEG, PNG, zip, video) — no benefit

compression package options:

- threshold — minimum response size to compress (default 1024 bytes)
- level — gzip level 1-9 (default 6, higher = smaller but slower)
- filter — custom function to decide what to compress

Important for streaming:

- Don't compress streaming responses with dynamic content
- Compression buffers response — incompatible with Server-Sent Events
- Can be disabled per-response with `res.set('Content-Encoding', 'identity')`

```

1  const compression = require('compression')
2
3  // Enable compression for all responses
4  app.use(compression({
5    level: 6,           // gzip level (1=fast/big, 9=slow/small, 6=balanced)
6    threshold: 1024,    // only compress if response > 1KB (default)
7    filter: (req, res) => {
8      // Don't compress if client doesn't support it
9      if (req.headers['x-no-compression']) return false
10
11      // Don't compress already compressed formats
12      const contentType = res.getHeader('Content-Type') || ''
13      if (contentType.includes('image/') || contentType.includes('video/')) {
14        return false
15      }
16
17      // Use compression's default filter for the rest
18      return compression.filter(req, res)
19    }
20  }))
21
22  // Disable compression for SSE (Server-Sent Events)
23  app.get('/events', (req, res) => {
24    res.setHeader('Content-Type', 'text/event-stream')
25    res.setHeader('Cache-Control', 'no-cache')
26    res.setHeader('Content-Encoding', 'identity') // disable compression!
27
28    const interval = setInterval(() => {

```

```

29    res.write(`data: ${JSON.stringify({ time: Date.now() })}\n\n`)

```

```
30     }, 1000)
31
32     req.on('close', () => clearInterval(interval))
33 })
34
35 // Test compression:
36 // curl -H 'Accept-Encoding: gzip' -I http://localhost:3000/api/products
37 // Content-Encoding: gzip (means it's compressed!)
```

■ Interview Tip:

Compression should be one of the first optimizations for any Express API. A 50KB JSON response compresses to ~8KB — 84% reduction. This directly reduces bandwidth costs and improves API response times for users on slow connections.

What is rate limiting in Express?

■ Answer

Rate limiting restricts how many requests a client can make in a time window. Protects against DDoS, brute force, and API abuse.

express-rate-limit package:

- Simple, widely used
- Can use Redis store for distributed (multi-server) rate limiting
- Adds X-RateLimit-* headers to responses

Key configuration:

- windowMs — time window in ms
- max — max requests per window
- store — custom store (Redis for multi-server)
- keyGenerator — what to rate limit by (default: IP)
- skip — skip certain requests

Different limits for different endpoints:

- Strict limits for auth (5 per 15min)
- Moderate for general API (100 per 15min)
- Generous for static content

```
1  const rateLimit = require('express-rate-limit')
2  const RedisStore = require('rate-limit-redis')
3  const redis = require('ioredis').createClient()
4
5  // Redis-backed store (works across multiple servers!)
6  const redisStore = new RedisStore({
7    sendCommand: (...args) => redis.call(...args)
8  })
9
10 // General API rate limit
11 const apiLimiter = rateLimit({
12   windowMs: 15 * 60 * 1000, // 15 minutes
13   max: 100, // 100 requests per window
14   store: redisStore, // shared across all servers
15   standardHeaders: true, // X-RateLimit-* headers
16   legacyHeaders: false,
17   message: { error: 'Too many requests, please try again later' },
18   keyGenerator: (req) => req.ip // rate limit by IP
19 })
20
21 // Strict auth rate limit (brute force protection)
22 const authLimiter = rateLimit({
23   windowMs: 60 * 60 * 1000, // 1 hour window
24   max: 5, // 5 attempts max
25   store: redisStore,
26   skipSuccessfulRequests: true, // don't count successful logins
27
28   message: { error: 'Too many failed attempts. Try again in 1 hour.' }
```

```
28  })
29
30  // Per-user rate limit (for authenticated endpoints)
31  const userLimiter = rateLimit({
32    windowMs: 60 * 1000,          // 1 minute
33    max: 30,                      // 30 requests per minute
34    keyGenerator: (req) => req.user?.id || req.ip // by user ID!
35  })
36
37  // Apply limits
38  app.use('/api/', apiLimiter)      // all API routes
39  app.post('/auth/login', authLimiter, loginCtrl) // strict login
40  app.post('/auth/register', authLimiter, registerCtrl)
41  app.use('/api/', authenticate, userLimiter) // per-user after auth
```

■ Interview Tip:

Without a Redis store, each server has its own rate limit counter. If you have 3 servers, a client can make 3x the allowed requests. Always use Redis store in production with multiple server instances.

What is cookie-parser? How to handle cookies?

Answer

cookie-parser parses Cookie header into req.cookies object. Without it, req.cookies is undefined and you'd have to parse manually.

Signed vs unsigned cookies:

- Unsigned — plain text, readable and modifiable by client
- Signed — HMAC signed, tampering detected by server
- Use signed for session IDs or any trusted data

Cookie security options:

- httpOnly — not accessible by JavaScript (XSS protection)
- secure — only sent over HTTPS
- sameSite — CSRF protection ('strict', 'lax', 'none')
- maxAge — expiry in milliseconds
- domain — which domain receives the cookie
- path — which paths the cookie is sent to

```
1  const cookieParser = require('cookie-parser')
2
3  // Parse unsigned AND signed cookies
4  app.use(cookieParser(process.env.COOKIE_SECRET)) // secret for signed cookies
5
6  // Set cookies
7  app.post('/auth/login', async (req, res) => {
8    const { accessToken, refreshToken } = await authService.login(req.body)
9
10   // Refresh token in httpOnly cookie
11   res.cookie('refreshToken', refreshToken, {
12     httpOnly: true, // not readable by JS
13     secure: process.env.NODE_ENV === 'production',
14     sameSite: 'strict', // CSRF protection
15     maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days
16     path: '/auth' // only sent to /auth routes
17   })
18
19   // Access token in regular cookie (JS needs to read it)
20   res.cookie('theme', 'dark', {
21     maxAge: 365 * 24 * 60 * 60 * 1000
22   })
23
24   res.json({ accessToken })
25 })
26
27 // Read cookies
28 app.get('/auth/refresh', (req, res) => {
29   const refreshToken = req.cookies.refreshToken
30   if (!refreshToken) return res.status(401).json({ error: 'No refresh token' })
31
32   // verify and issue new access token
```

```
32 })
33
34 // Signed cookies
35 res.cookie('userId', '123', { signed: true })
36 const userId = req.signedCookies.userId // '123' if valid, false if tampered
37
38 // Clear cookie (logout)
39 app.post('/auth/logout', (req, res) => {
40   res.clearCookie('refreshToken', { httpOnly: true, sameSite: 'strict', path: '/auth' })
41   res.json({ message: 'Logged out' })
42 })
```

■ Interview Tip:

When clearing a cookie, you MUST pass the same options (path, domain, sameSite, secure) that were used when setting it. If they don't match, the browser considers them different cookies and the original one remains.

Q27 What is express-session? Session management?

■ Answer

express-session creates server-side sessions — stores session data on the server and sends a session ID to the client as a cookie.

Session vs JWT:

- Sessions: data on server, ID in cookie — server must look up on each request
- JWT: data in token, stateless — no server lookup needed
- Sessions: easy to revoke (delete from store)
- JWT: can't revoke until expiry (need token blacklist for revocation)

Session stores (never use default memory store in production!):

- connect-redis — Redis (recommended)
- connect-mongo — MongoDB
- connect-pg-simple — PostgreSQL
- Default (in-memory) — leaks memory, doesn't work with multiple servers

Session fixation protection:

- Call `req.session.regenerate()` after login (generates new session ID)

```

1  const session = require('express-session')
2  const RedisStore = require('connect-redis')(session)
3  const redis = require('ioredis').createClient()
4
5  app.use(session({
6    secret: process.env.SESSION_SECRET, // 32+ random chars
7    resave: false, // don't save if not modified
8    saveUninitialized: false, // don't create empty sessions
9    store: new RedisStore({ // Redis store (required for production!)
10      client: redis,
11      prefix: 'session:',
12      ttl: 86400 // 24 hours
13    }),
14    cookie: {
15      httpOnly: true,
16      secure: process.env.NODE_ENV === 'production',
17      sameSite: 'strict',
18      maxAge: 24 * 60 * 60 * 1000 // 24 hours
19    },
20    name: 'sid' // don't use default connect.sid – reveals technology
21  }))
22
23  // Login – set session data
24  app.post('/auth/login', async (req, res) => {
25    const user = await authService.login(req.body)
26
27    // Session fixation protection – ALWAYS regenerate after login!
28    req.session.regenerate((err) => {

```

```

29    if (err) return next(err)

```

```

30 req.session.userId = user._id
31 req.session.role = user.role
32 req.session.save(() => res.json({ user }))
33 })
34 })
35
36 // Auth middleware using session
37 const authenticate = (req, res, next) => {
38   if (!req.session.userId) return res.status(401).json({ error: 'Not logged in' })
39   next()
40 }
41
42 // Logout
43 app.post('/auth/logout', (req, res) => {
44   req.session.destroy((err) => {
45     res.clearCookie('sid')
46     res.json({ message: 'Logged out' })
47   })
48 })

```

■ Interview Tip:

req.session.regenerate() after login is critical for security. Session fixation attacks work by giving the user a known session ID before login, then using that ID after they authenticate. Regenerating creates a fresh session ID that the attacker doesn't know.

How to implement JWT auth in Express?

■ Answer

JWT (JSON Web Token) authentication is the standard for stateless REST API auth.

Complete JWT flow:

- POST /auth/login — verify credentials → generate access + refresh tokens
- GET /api/* — send access token in Authorization: Bearer <token>
- POST /auth/refresh — use refresh token to get new access token
- POST /auth/logout — invalidate refresh token

Token storage strategy:

- Access token — in memory (React state) — short lived (15min)
- Refresh token — in HttpOnly cookie — long lived (7 days)
- This protects against both XSS and CSRF

Security considerations:

- Short access token expiry prevents long-term abuse
- Refresh token rotation — issue new refresh on every use
- Store refresh token hash in DB for revocation capability

```
1  const jwt = require('jsonwebtoken')
2  const bcrypt = require('bcrypt')
3
4  const ACCESS_SECRET = process.env.JWT_ACCESS_SECRET
5  const REFRESH_SECRET = process.env.JWT_REFRESH_SECRET
6
7  const signTokens = (userId, role) => ({
8    accessToken: jwt.sign({ userId, role }, ACCESS_SECRET, { expiresIn: '15m' }),
9    refreshToken: jwt.sign({ userId }, REFRESH_SECRET, { expiresIn: '7d' })
10 })
11
12 // Login
13 app.post('/auth/login', async (req, res, next) => {
14   try {
15     const { email, password } = req.body
16     const user = await User.findOne({ email }).select('+password')
17     if (!user || !await bcrypt.compare(password, user.password))
18       return res.status(401).json({ error: 'Invalid credentials' })
19
20     const { accessToken, refreshToken } = signTokens(user._id, user.role)
21     await Token.create({ userId: user._id, tokenHash: hashToken(refreshToken) })
22
23     res.cookie('refreshToken', refreshToken, {
24       httpOnly: true, secure: true, sameSite: 'strict',
25       maxAge: 7 * 24 * 60 * 60 * 1000
26     })
27     res.json({ accessToken, user: { id: user._id, name: user.name, role: user.role } })
28   } catch(err) { next(err) }
29 })
```

```

30
31 // Auth middleware
32 const authenticate = async (req, res, next) => {
33   const token = req.headers.authorization?.split(' ')[1]
34   if (!token) return res.status(401).json({ error: 'No token provided' })
35   try {
36     const decoded = jwt.verify(token, ACCESS_SECRET, { algorithms: ['HS256'] })
37     req.user = { id: decoded.userId, role: decoded.role }
38     next()
39   } catch(err) {
40     const code = err.name === 'TokenExpiredError' ? 'TOKEN_EXPIRED' : 'INVALID_TOKEN'
41     res.status(401).json({ error: 'Authentication failed', code })
42   }
43 }
44
45 // Refresh tokens
46 app.post('/auth/refresh', async (req, res, next) => {
47   const { refreshToken } = req.cookies
48   if (!refreshToken) return res.status(401).json({ error: 'No refresh token' })
49   try {
50     const decoded = jwt.verify(refreshToken, REFRESH_SECRET)
51     const stored = await Token.findOne({ userId: decoded.userId, tokenHash: hashToken(refreshToken) })
52     if (!stored) return res.status(401).json({ error: 'Invalid refresh token' })
53     const { accessToken } = signTokens(decoded.userId, (await User.findById(decoded.userId)).role)
54     res.json({ accessToken })
55   } catch(err) { next(err) }
56 })

```

■ Interview Tip:

Always specify algorithms: ['HS256'] when verifying JWTs. Without this, an attacker could change the algorithm to 'none' in the header, skip the signature, and forge any JWT. This is a well-known JWT vulnerability.

What is RBAC in Express?

■ Answer

RBAC (Role-Based Access Control) restricts access based on the user's role.

Implementation:

- User has a role (admin, editor, user, moderator)
- Each role has permitted actions
- Middleware checks role before allowing access

3 levels of authorization:

- Role check — is user's role in allowed roles?
- Permission check — does user have this permission?
- Resource ownership — does user own this resource?

Dynamic permissions:

- Store permissions in DB for runtime configuration
- Super admin can grant/revoke permissions
- Audit log all permission changes

Always authenticate before authorizing:

- `authenticate()` runs first — sets `req.user`
- `authorize()` runs second — checks `req.user.role`

```
1 // Role-based middleware
2 const authorize = (...allowedRoles) => (req, res, next) => {
3   if (!req.user) return res.status(401).json({ error: 'Not authenticated' })
4   if (!allowedRoles.includes(req.user.role)) {
5     return res.status(403).json({
6       error: 'Insufficient permissions',
7       required: allowedRoles,
8       current: req.user.role
9     })
10  }
11  next()
12 }
13
14 // Resource ownership check
15 const authorizeOwner = (getResource) => async (req, res, next) => {
16   const resource = await getResource(req)
17   if (!resource) return res.status(404).json({ error: 'Resource not found' })
18   // Admin bypasses ownership check
19   const isOwner = resource.userId.toString() === req.user.id
20   const isAdmin = req.user.role === 'admin'
21   if (!isOwner && !isAdmin) return res.status(403).json({ error: 'Forbidden' })
22   req.resource = resource
23   next()
24 }
25
```

```
26 // Permission-based (finer grained)
```

```

27 const ROLE_PERMISSIONS = {
28   admin: ['users:read', 'users:write', 'users:delete', 'posts:*', 'settings:write'],
29   editor: ['posts:read', 'posts:write', 'posts:publish'],
30   user: ['posts:read', 'profile:write']
31 }
32
33 const requirePermission = (permission) => (req, res, next) => {
34   const userPerms = ROLE_PERMISSIONS[req.user.role] || []
35   const hasPermission = userPerms.some(p =>
36     p === permission || p === permission.split(':')[0] + ':'
37   )
38   if (!hasPermission) return res.status(403).json({ error: `Missing: ${permission}` })
39   next()
40 }
41
42 // Usage
43 router.get('/admin/users', authenticate, authorize('admin'), userCtrl.listAll)
44 router.put('/posts/:id', authenticate, authorize('admin', 'editor'), postCtrl.update)
45 router.delete('/posts/:id', authenticate,
46   authorizeOwner(req => Post.findById(req.params.id)), postCtrl.delete)
47 router.post('/posts/publish', authenticate, requirePermission('posts:publish'), postCtrl.publish)

```

■ Interview Tip:

RBAC is simpler to implement and understand than ABAC. Start with RBAC for most apps. Only move to ABAC (attribute-based) when you need complex rules like 'users in department X can only view data from their region during business hours'.

Q30 How to prevent SQL/NoSQL injection in Express?

■ Answer

Injection attacks embed malicious code in inputs that get executed by the database.

NoSQL (MongoDB) injection:

- Attacker sends JSON operators: { password: { '\$ne': '' } }
- Bypasses authentication checks
- Prevention: express-mongo-sanitize, Mongoose type validation, Zod

SQL injection:

- Attacker sends: ' OR 1=1 --
- Prevention: ALWAYS use parameterized queries

express-mongo-sanitize:

- Strips \$ and . from req.body, req.query, req.params
- Prevents MongoDB operator injection

Defense in depth:

- Input validation (Zod) — type check first
- Sanitization (mongoSanitize) — strip dangerous chars
- Parameterized queries — never concatenate user input

```
1  const mongoSanitize = require('express-mongo-sanitize')
2  const { z } = require('zod')
3
4  // Middleware: sanitize ALL request input
5  app.use(mongoSanitize({
6    replaceWith: '_', // replace $ with _ instead of removing
7    onSanitize: ({ req, key }) => {
8      console.warn(`Sanitized: ${key} in ${req.url}`) // log suspicious input
9    }
10 })))
11
12 // ■ VULNERABLE: direct MongoDB query with user input
13 app.post('/auth/login', async (req, res) => {
14   const user = await User.findOne({
15     email: req.body.email,
16     password: req.body.password // ← NEVER QUERY BY RAW PASSWORD
17     // Attacker sends: { password: { '$ne': '' } }
18     // Matches ANY user with non-empty password!
19   })
20 })
21
22 // ■ SAFE: validate types + never query by raw password
23 const loginSchema = z.object({
24   body: z.object({
25     email: z.string().email(), // must be valid email format
26     password: z.string().min(1) // must be plain string
```

```
27  })
```

```

28 })
29
30 app.post('/auth/login', validate(loginSchema), async (req, res) => {
31   // Find by email only
32   const user = await User.findOne({ email: req.body.email }).select('+password')
33   if (!user) return res.status(401).json({ error: 'Invalid credentials' })
34
35   // Compare password with bcrypt (safe!)
36   const valid = await bcrypt.compare(req.body.password, user.password)
37   if (!valid) return res.status(401).json({ error: 'Invalid credentials' })
38   // ← bcrypt compare always gets string, not MongoDB operator
39 })
40
41 // ■ SAFE: Parameterized query (PostgreSQL)
42 const { rows } = await pool.query(
43   'SELECT * FROM users WHERE email = $1 AND active = true',
44   [req.body.email] // ← $1 is parameterized, never concatenated
45 )

```

■ Interview Tip:

The safest login pattern: never query MongoDB by raw password. Find user by email, then compare with `bcrypt.compare()`. This completely eliminates NoSQL injection on the auth endpoint because we never put `req.body.password` into a MongoDB query.

How to handle file uploads in Express?

■ Answer

Multer is the standard Express middleware for handling multipart/form-data.

Multer storage options:

- `memoryStorage()` — file as Buffer in memory (process before saving)
- `diskStorage()` — save to disk with custom destination and filename

File validation (critical for security!):

- `fileFilter` — reject non-allowed MIME types
- `limits.fileSize` — max file size (bytes)
- NEVER trust file extension — check actual MIME type

Production approach:

- `multer.memoryStorage()` + process (resize, validate) + upload to S3
- Never store files on server disk permanently (horizontally scaled servers don't share filesystem)

Common options:

- `upload.single('fieldname')` — one file
- `upload.array('fieldname', max)` — multiple files
- `upload.fields([name, maxCount])` — multiple fields

```

1  const multer = require('multer')
2  const sharp = require('sharp') // image processing
3  const { S3Client, PutObjectCommand } = require('@aws-sdk/client-s3')
4  const crypto = require('crypto')
5
6  const s3 = new S3Client({ region: process.env.AWS_REGION })
7
8  // Memory storage — process before saving to S3
9  const upload = multer({
10   storage: multer.memoryStorage(),
11   limits: {
12     fileSize: 10 * 1024 * 1024, // 10MB max
13     files: 5 // max 5 files at once
14   },
15   fileFilter: (req, file, cb) => {
16     const allowedMimes = new Set(['image/jpeg', 'image/png', 'image/webp', 'image/gif'])
17     if (allowedMimes.has(file.mimetype)) cb(null, true)
18     else cb(new Error(`${file.mimetype} not allowed. Use JPEG, PNG, WebP, GIF`), false)
19   }
20 })
21
22 async function uploadToS3(buffer, mimetype, folder) {
23   const ext = mimetype.split('/')[1]
24   const key = `${folder}/${crypto.randomBytes(16).toString('hex')}.${ext}`
25   await s3.send(new PutObjectCommand({
26     Bucket: process.env.S3_BUCKET, Key: key,

```

```

27   Body: buffer, ContentType: mimetype,

```

```

28   CacheControl: 'max-age=31536000' // 1 year browser cache
29   )))
30   return `https://${process.env.CDN_DOMAIN}/${key}` // CloudFront URL
31 }
32
33 // Avatar upload endpoint
34 router.post('/users/avatar', authenticate,
35   upload.single('avatar'),
36   async (req, res, next) => {
37     if (!req.file) return res.status(400).json({ error: 'No file provided' })
38
39     // Process: resize + convert to WebP
40     const processed = await sharp(req.file.buffer)
41       .resize(200, 200, { fit: 'cover', position: 'center' })
42       .webp({ quality: 85 })
43       .toBuffer()
44
45     const avatarUrl = await uploadToS3(processed, 'image/webp', 'avatars')
46     await User.findByIdAndUpdate(req.user.id, { avatar: avatarUrl })
47     res.json({ avatar: avatarUrl })
48   }
49 )

```

■ Interview Tip:

Always resize and convert images server-side before storing. A user uploading a 15MB DSLR photo to be used as a 200x200 avatar is wasted storage. Use sharp to resize, and convert to WebP for ~30% smaller files than JPEG at same quality.

What is HTTP security headers? How to implement?

Answer

HTTP security headers protect against many common attacks.

helmet.js sets these automatically, but knowing what they do helps you configure correctly.

Critical security headers:

- Content-Security-Policy — whitelist scripts, styles, images sources
- Strict-Transport-Security (HSTS) — force HTTPS forever
- X-Content-Type-Options: nosniff — prevent MIME type sniffing
- X-Frame-Options — prevent embedding in iframes (clickjacking)
- Referrer-Policy — control what referrer is sent
- Permissions-Policy — disable browser features (camera, mic, geolocation)

What attacks they prevent:

- CSP → XSS
- HSTS → SSL stripping
- nosniff → content type confusion
- X-Frame-Options → clickjacking

Verify headers with:

- securityheaders.com — grade your headers
- Mozilla Observatory — comprehensive security check

```
1  const helmet = require('helmet')
2  const crypto = require('crypto')
3
4  // Generate nonce for inline scripts (per-request)
5  app.use((req, res, next) => {
6    res.locals.nonce = crypto.randomBytes(16).toString('base64')
7    next()
8  })
9
10 app.use((req, res, next) => {
11   helmet({
12     contentSecurityPolicy: {
13       directives: {
14         defaultSrc: ['self'],
15         scriptSrc: ['self', `nonce-${res.locals.nonce}`],
16         styleSrc: ['self', 'unsafe-inline'],
17         imgSrc: ['self', 'data:', '*.cloudfront.net'],
18         connectSrc: ['self', 'api.myapp.com'],
19         fontSrc: ['self', 'fonts.gstatic.com'],
20         objectSrc: ['none'],
21         frameAncestors: ['self'], // can't embed in iframes
22       }
23     },
24     hsts: { maxAge: 63072000, includeSubDomains: true, preload: true },
25   })(req, res, next)
```

```
26 }
```

```

27
28 // Additional custom headers
29 app.use((req, res, next) => {
30   // CORP – cross-origin resource policy
31   res.setHeader('Cross-Origin-Resource-Policy', 'same-origin')
32   // COOP – cross-origin opener policy
33   res.setHeader('Cross-Origin-Opener-Policy', 'same-origin')
34   // Permissions policy
35   res.setHeader('Permissions-Policy',
36     'accelerometer=(), camera=(), geolocation=(), gyroscope=(), microphone=()'
37   )
38   next()
39 })
40
41 // Cache control for API responses
42 app.use('/api/', (req, res, next) => {
43   res.setHeader('Cache-Control', 'no-store, no-cache, must-revalidate')
44   res.setHeader('Pragma', 'no-cache')
45   next()
46 })

```

■ Interview Tip:

Run your API through securityheaders.com to get a free security grade. Most APIs start at C or D. With proper helmet configuration you can reach A+ quickly. Security auditors and enterprise clients often check this score.

How to implement pagination in Express?

■ Answer

Pagination is essential for any endpoint that returns a list of resources.

Two main strategies:

- Offset pagination — skip N records. Simple but slow at large offsets.
- Cursor pagination — use last item ID. Consistent performance, no duplicates.

Offset pagination response structure:

```
{ data, pagination: { page, limit, total, totalPages, hasNext, hasPrev } }
```

Cursor pagination response structure:

```
{ data, nextCursor, hasMore }
```

Performance tip:

- Run data query and count query in parallel with Promise.all
- Use .lean() for read-only queries (2-3x faster)
- Index the sorted fields

```

1 // Offset pagination
2 router.get('/products', async (req, res, next) => {
3   try {
4     const page = Math.max(1, parseInt(req.query.page) || 1)
5     const limit = Math.min(100, Math.max(1, parseInt(req.query.limit) || 20))
6     const skip = (page - 1) * limit
7
8     const filter = buildFilter(req.query) // your filter logic
9     const sortField = ['price', 'name', 'createdAt'].includes(req.query.sort)
10      ? req.query.sort : 'createdAt'
11     const sortOrder = req.query.order === 'asc' ? 1 : -1
12
13     // Run count + data queries in PARALLEL
14     const [data, total] = await Promise.all([
15       Product.find(filter).sort({ [sortField]: sortOrder })
16         .skip(skip).limit(limit).lean(),
17       Product.countDocuments(filter)
18     ])
19
20     res.json({
21       data,
22       pagination: {
23         page, limit, total,
24         totalPages: Math.ceil(total / limit),
25         hasNext: page * limit < total,
26         hasPrev: page > 1,
27         from: skip + 1,
28         to: Math.min(skip + limit, total)
29       }
30     })

```

```

31 } catch(err) { next(err) }

```

```

32 })
33
34 // Cursor pagination (for infinite scroll)
35 router.get('/feed', async (req, res, next) => {
36   try {
37     const limit = 20
38     const cursor = req.query.cursor // last item's _id
39
40     const filter = cursor
41       ? { _id: { $lt: cursor }, ...baseFilter } // items before cursor
42       : baseFilter
43
44     const items = await Post.find(filter)
45       .sort({ _id: -1 })
46       .limit(limit + 1) // fetch 1 extra to detect hasMore
47       .lean()
48
49     const hasMore = items.length > limit
50     const data = hasMore ? items.slice(0, limit) : items
51     const nextCursor = hasMore ? data[data.length - 1]._id : null
52
53     res.json({ data, nextCursor, hasMore })
54   } catch(err) { next(err) }
55 })

```

■ Interview Tip:

The limit+1 trick for cursor pagination is elegant: fetch one more item than needed. If you get limit+1 items back, there's more data (hasMore=true). Return only the first limit items. This avoids a separate COUNT query.

How to implement search in Express?

■ Answer

Search endpoints combine text matching, filtering, and sorting.

Search implementation approaches:

- MongoDB \$text search — built-in, requires text index
- MongoDB \$regex — flexible but slow without index
- Elasticsearch — full-featured, scalable, separate service
- Algolia — hosted search-as-a-service

MongoDB text search setup:

- Create text index on searchable fields
- Use \$text: { \$search: 'query' } to search
- Sort by relevance with { score: { \$meta: 'textScore' } }

Search with highlighting:

- Regex-based highlighting for matched terms
- Elasticsearch has built-in highlight feature

Debouncing on frontend:

- Search API called after user stops typing (300-500ms delay)
- Reduces unnecessary API calls

```
1 // Text index setup in Mongoose model
2 // productSchema.index({ name: 'text', description: 'text', tags: 'text' },
3 //   { weights: { name: 10, tags: 5, description: 1 } })
4
5 router.get('/search', async (req, res, next) => {
6   try {
7     const { q: query, category, page = 1, limit = 20 } = req.query
8
9     if (!query || query.trim().length < 2) {
10       return res.status(400).json({ error: 'Search query must be at least 2 chars' })
11     }
12
13     const sanitized = query.trim().substring(0, 100)
14     const skip = (parseInt(page) - 1) * parseInt(limit)
15
16     const filter = {
17       $text: { $search: sanitized },
18       ...(category && { category })
19     }
20
21     const [results, total] = await Promise.all([
22       Product.find(filter, { score: { $meta: 'textScore' } })
23         .sort({ score: { $meta: 'textScore' }, createdAt: -1 })
24         .skip(skip)
25         .limit(parseInt(limit))
26         .lean(),
```

```

27     Product.countDocuments(filter)
28   ])
29
30   res.json({
31     query: sanitized,
32     results,
33     pagination: {
34       page: parseInt(page), total,
35       totalPages: Math.ceil(total / parseInt(limit))
36     }
37   })
38 } catch(err) { next(err) }
39 })
40
41 // Autocomplete – fast prefix search with regex
42 router.get('/search/autocomplete', async (req, res, next) => {
43   const q = req.query.q?.trim()
44   if (!q || q.length < 1) return res.json([])
45
46   // Limit to 5 results, only return name field
47   const suggestions = await Product.find(
48     { name: { $regex: `^${escapeRegex(q)}`, $options: 'i' } },
49     { name: 1, category: 1 } // projection
50   ).limit(5).lean()
51
52   res.json(suggestions)
53 })
54
55 // Escape regex special characters
56 function escapeRegex(str) {
57   return str.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]\]/g, '\\$&')
58 }

```

■ Interview Tip:

Always escape regex special characters in user-provided search strings. If a user types '.' or '.*' in a regex search, it matches everything — potential denial of service and unexpected results.

Q35 How to handle async errors properly in Express?

■ Answer

Express 4 does NOT automatically catch async errors — you must handle them explicitly.

The problem:

- Async route handler throws → becomes unhandled promise rejection
- Does NOT reach the global error handler
- App continues running with silent failure

Solutions:

- asyncHandler wrapper — automatically pass async errors to next()
- try/catch in every route — verbose but explicit
- Express 5 — handles async automatically (still in beta)

asyncHandler pattern:

- Wrap route handlers with a HOF
- Any thrown error or rejected promise → next(err)
- Error reaches global error handler

Global vs local error handling:

- Global: catch all, return consistent format
- Local: catch specific errors, add context

```
1 // asyncHandler — the essential wrapper
2 const asyncHandler = (fn) => (req, res, next) => {
3   Promise.resolve(fn(req, res, next)).catch(next)
4 }
5 // OR using async function directly:
6 const asyncHandler2 = (fn) => async (req, res, next) => {
7   try { await fn(req, res, next) }
8   catch(err) { next(err) }
9 }
10
11 // ■ Without asyncHandler — error silently swallowed!
12 app.get('/bad', async (req, res) => {
13   const data = await fetchData() // throws
14   // ← Error does NOT reach global handler — unhandled rejection!
15   res.json(data)
16 })
17
18 // ■ With asyncHandler — error reaches global handler
19 app.get('/good', asyncHandler(async (req, res) => {
20   const data = await fetchData() // throws
21   // asyncHandler catches it and calls next(err)
22   res.json(data)
23 })))
24
25 // ■ With try/catch (explicit, good for complex error handling)
```

```
26 app.get('/explicit', async (req, res, next) => {
```

```

27   try {
28     const user = await User.findById(req.params.id)
29     if (!user) {
30       // Add context before passing to error handler
31       const err = new NotFoundError('User')
32       err.userId = req.params.id // extra context for logging
33       return next(err)
34     }
35     res.json(user)
36   } catch(err) {
37     next(err)
38   }
39 })
40
41 // Express 5 (beta) – handles async natively
42 // app.use(require('express').Router()) // v5 catches async errors!
43 // app.get('/v5', async (req, res) => {
44 //   const data = await fetchData() // auto-caught if throws
45 //   res.json(data)
46 // })

```

■ Interview Tip:

The `asyncHandler` wrapper should be used on EVERY async route in Express 4. A common team approach: import `asyncHandler` at the top of every route file and wrap all route handlers by default. Make it the standard, not the exception.

How to implement request timeout in Express?

■ Answer

Request timeouts prevent slow endpoints from hanging connections indefinitely.

Why timeouts matter:

- External service hangs → your endpoint hangs → connections exhaust
- Attack: slowly send request body → hold connections (Slowloris DDoS)
- Without timeout: connections pile up until server crashes

Timeout approaches:

- `server.setTimeout()` — sets server-level socket timeout
- `connect-timeout` middleware — per-request timeout
- Axios/Fetch options — timeout for outgoing requests
- `AbortController` — cancel specific operations

What to do on timeout:

- Send 503 Service Unavailable
- Log the slow endpoint for optimization
- Abort any in-flight operations if possible

```
1  const timeout = require('connect-timeout')
2
3  // Server-level timeout (2 minutes for long-running requests)
4  const server = app.listen(3000)
5  server.setTimeout(120000) // 2 minutes
6
7  // Request-level timeout middleware
8  function requestTimeout(duration = '30s') {
9    return [
10     timeout(duration),
11     (req, res, next) => {
12       if (req.timedout) {
13         return res.status(503).json({
14           error: 'Request timeout',
15           timeout: duration,
16           path: req.path
17         })
18       }
19       next()
20     }
21   ]
22 }
23
24 // Apply different timeouts to different routes
25 app.get('/api/quick', requestTimeout('5s'), quickController)
26 app.post('/api/export', requestTimeout('60s'), exportController)
27
28 // Timeout for outgoing HTTP requests
```

```
29 const axios = require('axios').create({ timeout: 5000 }) // 5s max
```

```

30 // Or with AbortController:
31 async function fetchWithTimeout(url, timeoutMs = 5000) {
32   const controller = new AbortController()
33   const timer = setTimeout(() => controller.abort(), timeoutMs)
34   try {
35     const res = await fetch(url, { signal: controller.signal })
36     return await res.json()
37   } catch(err) {
38     if (err.name === 'AbortError') throw new Error(`Request to ${url} timed out`)
39     throw err
40   } finally {
41     clearTimeout(timer)
42   }
43 }
44
45 // Database query timeout
46 const user = await User.findById(id).maxTimeMS(5000) // MongoDB 5s timeout
47 const result = await pool.query('SELECT * FROM users', [], { timeout: 5000 })

```

■ Interview Tip:

Set timeouts for all outgoing requests. If an external API hangs and you don't have a timeout, your request handler waits indefinitely, holding up the Node.js connection. In high-traffic scenarios, this cascades into all connections being held and your server appearing down.

Q37

What is helmet and express-mongo-sanitize together?

■ Answer

Combining helmet and express-mongo-sanitize provides strong defense against common attacks.

Combined security middleware stack:

- helmet — HTTP security headers
- express-mongo-sanitize — NoSQL injection prevention
- cors — cross-origin control
- express-rate-limit — DDoS/brute force protection
- hpp — HTTP Parameter Pollution prevention
- compression — gzip (performance, not security but essential)

HTTP Parameter Pollution (hpp):

- Request: ?sort=price&sort=name
- Without hpp: req.query.sort = ['price', 'name'] (array!)
- Code expecting string gets array → crash
- hpp ensures single value for each param

Security middleware order matters:

1. helmet first (security headers on all responses)
2. cors (allows cross-origin)
3. body parsers
4. sanitization
5. rate limiting
6. authentication
7. routes

```
1 const express = require('express')
2 const helmet = require('helmet')
3 const cors = require('cors')
4 const mongoSanitize = require('express-mongo-sanitize')
5 const hpp = require('hpp')
6 const rateLimit = require('express-rate-limit')
7 const compression = require('compression')
8
9 const app = express()
10
11 // 1. Security headers (first!)
12 app.use(helmet())
13
14 // 2. CORS
15 app.use(cors({ origin: process.env.ALLOWED_ORIGINS.split(','), credentials: true }))
16
17 // 3. Compression
18 app.use(compression())
19
20 // 4. Body parsing with size limits
21 app.use(express.json({ limit: '10kb' })))
```

```
22 app.use(express.urlencoded({ extended: true, limit: '10kb' })))
```



```
23
24 // 5. NoSQL injection prevention
25 app.use(mongoSanitize())
26
27 // 6. HTTP Parameter Pollution prevention
28 app.use(hpp({
29   whitelist: ['price', 'rating', 'tags'] // these can be arrays (intentional)
30 })))
31
32 // 7. Rate limiting
33 app.use('/api/', rateLimit({ windowMs: 15*60*1000, max: 100 })))
34
35 // 8. Routes come LAST
36 app.use('/api/v1', require('./routes'))
37
38 // Result: your app is protected against:
39 // ■ XSS (Content-Security-Policy via helmet)
40 // ■ Clickjacking (X-Frame-Options via helmet)
41 // ■ HTTPS enforcement (HSTS via helmet)
42 // ■ NoSQL injection (express-mongo-sanitize)
43 // ■ DDoS/brute force (express-rate-limit)
44 // ■ Parameter pollution (hpp)
45 // ■ CORS attacks (cors with whitelist)
```

■ Interview Tip:

This security middleware stack should be the boilerplate for every production Express app. Copy-paste this into your project setup. hpp is underused but important — without it, any array parameter crashes code expecting a string.

What is the MVC pattern in Express?

■ Answer

MVC (Model-View-Controller) organizes code by responsibility for maintainability.

Express MVC layers:

- Routes — URL mapping and middleware chain
- Controllers — handle req/res, call services, return responses
- Services — business logic, call models/external APIs
- Models — data schema, DB queries

Benefits of MVC:

- Each file has ONE job
- Controllers are thin — only HTTP concerns
- Services are testable without HTTP
- Models encapsulate data access

Request flow:

Request → Router → Middleware → Controller → Service → Model → DB

Response ← Controller ← Service ← Model ← DB

Rule of thumb:

- Controller: < 20 lines per handler
- Service: business logic, no req/res objects
- Model: database operations only

```
1 // models/User.js — data layer
2 const User = mongoose.model('User', userSchema)
3 module.exports = User
4
5 // services/userService.js — business logic
6 const User = require('../models/User')
7 const emailService = require('./emailService')
8
9 const userService = {
10   async register(data) {
11     const exists = await User.findOne({ email: data.email })
12     if (exists) throw new ConflictError('Email already registered')
13
14     const user = await User.create(data) // password hashed by Mongoose hook
15     await emailService.sendWelcome(user.email, user.name) // side effect
16     return user
17   },
18
19   async getById(id) {
20     const user = await User.findById(id).lean()
21     if (!user) throw new NotFoundError('User')
22     return user
23   },
24 }
```

```

25   async update(id, changes, requesterId) {
26       if (id !== requesterId.toString()) throw new ForbiddenError()
27       return User.findByIdAndUpdate(id, changes, { new: true, runValidators: true })
28   }
29 }
30 module.exports = userService
31
32 // controllers/userController.js – HTTP layer
33 const userService = require('../services/userService')
34
35 const userController = {
36     register: asyncHandler(async (req, res) => {
37         const user = await userService.register(req.body) // no business logic here
38         res.status(201).json({ user })
39     }),
40
41     getById: asyncHandler(async (req, res) => {
42         const user = await userService.getById(req.params.id)
43         res.json({ user })
44     }),
45 }
46 module.exports = userController
47
48 // routes/users.js – routing layer
49 const router = require('express').Router()
50 const ctrl = require('../controllers/userController')
51 router.post('/', validate(registerSchema), ctrl.register)
52 router.get('/:id', authenticate, ctrl.getById)
53 module.exports = router

```

■ Interview Tip:

Keep controllers thin. If your controller is doing: validate input, query database, transform data, send email, update cache, log audit — it's doing too much. The controller should only orchestrate: call service → transform result → send response.

What is dependency injection in Express?

■ Answer

Dependency injection (DI) passes dependencies into functions instead of creating them inside.

Why DI in Express:

- Testability — inject mocks in tests
- Flexibility — swap implementations
- Explicit dependencies — easier to understand

Patterns in Node.js:

- Constructor injection — pass deps to class constructor
- Factory functions — return service with injected deps
- Container — automatic dependency resolution (Awilix, Inversify)

Without DI (hard to test):

- Service creates its own DB connection
- Can't mock the DB in unit tests

With DI (easy to test):

- Inject DB dependency from outside
- In tests: inject mock DB
- In production: inject real DB

```
1 // ■ Without DI — hard to test
2 class UserService {
3   async getUser(id) {
4     const db = require('../db') // hidden dependency!
5     return db.users.findById(id) // can't mock in tests
6   }
7 }
8
9 // ■ With DI — testable
10 class UserService {
11   constructor({ userRepository, emailService, logger }) {
12     this.userRepo = userRepository // injected!
13     this.emailService = emailService // injected!
14     this.logger = logger // injected!
15   }
16
17   async register(data) {
18     const exists = await this.userRepo.findByEmail(data.email)
19     if (exists) throw new ConflictError('Email exists')
20     const user = await this.userRepo.create(data)
21     await this.emailService.sendWelcome(user.email)
22     this.logger.info({ userId: user._id }, 'User registered')
23     return user
24   }
25 }
```

```
27 // Composition root – wire everything together
28 // src/container.js
29 const userRepo = new MongoUserRepository(db)
30 const emailService = new SendGridEmailService(process.env.SENDGRID_KEY)
31 const logger = require('./logger')
32 const userService = new UserService({ userRepo, emailService, logger })
33 const userController = new UserController({ userService })
34
35 // Test – inject mocks easily
36 const mockRepo = { findByEmail: jest.fn(), create: jest.fn() }
37 const mockEmail = { sendWelcome: jest.fn() }
38 const mockLogger = { info: jest.fn(), error: jest.fn() }
39 const testService = new UserService({
40   userRepository: mockRepo,
41   emailService: mockEmail,
42   logger: mockLogger
43 })
44 // Now test without real DB or email service!
```

■ Interview Tip:

DI makes unit testing trivial. With proper injection, you can test your entire business logic without a real database, email service, or external APIs. Just inject mock objects that return predefined values. Tests run in milliseconds.

What is the repository pattern in Express?

Answer

The repository pattern abstracts the data access layer from business logic.

Benefits:

- Business logic doesn't care where data comes from (MongoDB, PostgreSQL, API, cache)
- Easy to swap databases
- Testable — inject mock repositories
- Single place for all queries related to an entity

Repository interface:

- `findById(id)` — get one by ID
- `findAll(filter, options)` — get many
- `create(data)` — create new
- `update(id, changes)` — modify
- `delete(id)` — remove
- Custom: `findByEmail(email)`, `findActiveByRole(role)`

Repository vs Model:

- Mongoose Model already IS a repository in simple apps
- Use separate repository class when you need DB-agnostic code

```
1 // repositories/UserRepository.js
2 class MongoUserRepository {
3   constructor(UserModel) {
4     this.User = UserModel // injected Mongoose model
5   }
6
7   async findById(id) {
8     return this.User.findById(id).lean()
9   }
10
11  async findByEmail(email) {
12    return this.User.findOne({ email: email.toLowerCase() }).lean()
13  }
14
15  async findActive(filter = {}, { page = 1, limit = 20, sort = 'createdAt' } = {}) {
16    const skip = (page - 1) * limit
17    const [data, total] = await Promise.all([
18      this.User.find({ ...filter, active: true })
19        .sort({ [sort]: -1 }).skip(skip).limit(limit).lean(),
20      this.User.countDocuments({ ...filter, active: true })
21    ])
22    return { data, total, page, limit }
23  }
24
25  async create(data) {
26    const user = new this.User(data)
27    return user.save()
```

```

28 }
29
30 async update(id, changes) {
31   return this.User.findByIdAndUpdate(id, { $set: changes }, { new: true, runValidators: true })
32 }
33
34 async softDelete(id) {
35   return this.User.findByIdAndUpdate(id, { $set: { active: false, deletedAt: new Date() } })
36 }
37 }
38
39 // Wire up
40 const User = require('./models/User')
41 const userRepo = new MongoUserRepository(User)
42
43 // Service uses repo, not model directly
44 class UserService {
45   constructor(userRepo) { this.userRepo = userRepo }
46   async getUser(id) { return this.userRepo.findById(id) }
47 }
48
49 // Test with mock repository
50 const mockRepo = {
51   findById: jest.fn().mockResolvedValue({ id: '1', name: 'Test' }),
52   findByEmail: jest.fn().mockResolvedValue(null),
53   create: jest.fn().mockImplementation(data => Promise.resolve({ ...data, _id: '1' }))
54 }

```

■ Interview Tip:

Soft delete (setting active:false instead of deleting) is a best practice for production. You can recover accidentally deleted data. The Mongoose pre-find hook filters out inactive users, so they never appear in queries.

Q41 How to implement WebSockets alongside Express?

■ Answer

WebSockets provide real-time bidirectional communication over a persistent connection. Express serves HTTP; a WebSocket server runs alongside it.

Two approaches:

- ws library — lightweight, low-level, full control
- Socket.io — higher-level, rooms/namespaces, automatic fallback

Sharing port with Express:

- Create http.Server with Express app
- Attach WebSocket server to the http.Server
- Both HTTP and WebSocket on same port

Auth for WebSockets:

- Can't set custom headers in browser WebSocket API
- Pass token in query param: ws://api.app.com?token=xxx
- Or use cookie-based auth (cookies sent automatically)

Scaling WebSockets:

- Multiple servers: use Redis pub/sub for message broadcast

```
1  const http = require('http')
2  const { WebSocketServer } = require('ws')
3  const express = require('express')
4  const jwt = require('jsonwebtoken')
5
6  const app = express()
7  app.use(express.json())
8
9  // Create HTTP server (shared by Express and WebSocket)
10 const server = http.createServer(app)
11 const wss = new WebSocketServer({ server })
12
13 // Client map: userId → WebSocket
14 const clients = new Map()
15
16 wss.on('connection', (ws, req) => {
17   // Authenticate WebSocket connection
18   const url = new URL(req.url, 'http://localhost')
19   const token = url.searchParams.get('token')
20
21   if (!token) return ws.close(1008, 'No token') // 1008 = policy violation
22
23   let userId
24   try {
25     const decoded = jwt.verify(token, process.env.JWT_SECRET)
26     userId = decoded.userId
27   } catch {
```

```

28   return ws.close(1008, 'Invalid token')
29 }
30
31 clients.set(userId, ws)
32 ws.send(JSON.stringify({ type: 'CONNECTED', userId }))
33
34 ws.on('message', (data) => {
35   try {
36     const msg = JSON.parse(data.toString())
37     handleMessage(userId, msg, ws)
38   } catch { ws.send(JSON.stringify({ type: 'ERROR', error: 'Invalid JSON' })) }
39 })
40
41 ws.on('close', () => clients.delete(userId))
42 ws.on('error', (err) => { console.error('WS error:', err); clients.delete(userId) })
43 })
44
45 // Send to specific user from HTTP route
46 function sendToUser(userId, data) {
47   const ws = clients.get(userId)
48   if (ws?.readyState === ws.OPEN) {
49     ws.send(JSON.stringify(data))
50     return true
51   }
52   return false // user offline
53 }
54
55 // Broadcast to all connected clients
56 function broadcast(data) {
57   const msg = JSON.stringify(data)
58   wss.clients.forEach(ws => { if (ws.readyState === ws.OPEN) ws.send(msg) })
59 }
60
61 // Trigger WebSocket notification from REST endpoint
62 app.post('/orders', authenticate, asyncHandler(async (req, res) => {
63   const order = await Order.create({ ...req.body, userId: req.user.id })
64   sendToUser(req.user.id, { type: 'ORDER_CREATED', order }) // real-time!
65   res.status(201).json(order)
66 })))
67
68 server.listen(3000)

```

■ Interview Tip:

Use the same server for HTTP and WebSocket (port sharing). This simplifies deployment — only one port to expose in firewall/Kubernetes. The ws library is perfect for simple real-time features; use Socket.io when you need rooms, namespaces, and automatic reconnection.

What is Server-Sent Events (SSE) in Express?

■ Answer

SSE is a simple way to push data from server to browser over HTTP. Unlike WebSocket, SSE is one-directional (server → client only).

SSE vs WebSocket:

- SSE: one-way, HTTP-based, auto-reconnect, simpler
- WebSocket: bidirectional, own protocol, more complex
- SSE: great for: live updates, notifications, progress
- WebSocket: great for: chat, games, collaborative editing

SSE format:

- Content-Type: text/event-stream
- Each event: data: <json>\n\n
- Named events: event: <type>\ndata: <json>\n\n
- Keep-alive: : comment\n\n

Browser auto-reconnects!

- If connection drops, browser reconnects automatically
- Sends Last-Event-ID header to resume from where it left off

```
1 // SSE endpoint
2 app.get('/api/events', authenticate, (req, res) => {
3   // Set SSE headers
4   res.setHeader('Content-Type', 'text/event-stream')
5   res.setHeader('Cache-Control', 'no-cache, no-transform')
6   res.setHeader('Connection', 'keep-alive')
7   res.setHeader('X-Accel-Buffering', 'no') // disable Nginx buffering
8   res.flushHeaders() // send headers immediately
9
10  const userId = req.user.id
11
12  // Send initial connection event
13  res.write(`event: connected\ndata: ${JSON.stringify({ userId })}\n\n`)
14
15  // Keep-alive ping every 30 seconds
16  const keepAlive = setInterval(() => {
17    res.write(': ping\n\n') // comment line = keep-alive
18  }, 30000)
19
20  // Subscribe to events for this user
21  const unsubscribe = eventBus.subscribe(userId, (event) => {
22    res.write(`id: ${event.id}\n`) // event ID for resume
23    res.write(`event: ${event.type}\n`) // event type
24    res.write(`data: ${JSON.stringify(event.data)}\n\n`) // data
25  })
26
27  // Cleanup when client disconnects
```

```
28 req.on('close', () => {
```

```

29     clearInterval(keepAlive)
30     unsubscribe()
31     console.log(`SSE client ${userId} disconnected`)
32   })
33 })
34
35 // Send events from other routes
36 app.post('/orders', authenticate, asyncHandler(async (req, res) => {
37   const order = await Order.create(req.body)
38
39   // Emit to SSE clients
40   eventBus.emit(req.user.id, {
41     id: Date.now().toString(),
42     type: 'ORDER_UPDATE',
43     data: { orderId: order._id, status: order.status }
44   })
45
46   res.status(201).json(order)
47 }))
48
49 // Browser side:
50 // const es = new EventSource('/api/events', { withCredentials: true })
51 // es.addEventListener('ORDER_UPDATE', e => console.log(JSON.parse(e.data)))

```

■ Interview Tip:

SSE is simpler than WebSocket for read-only dashboards and notification systems. The browser's EventSource API handles reconnection automatically. For scaling, store events in Redis pub/sub so all server instances can push to their connected clients.

Q43

How to handle multipart form data and file validation?

■ Answer

Multipart form data handles both file uploads and regular form fields in one request.

Security checklist for file uploads:

- Validate MIME type (not extension — easily spoofed!)
- Set maximum file size limit
- Rename files (prevent path traversal attacks)
- Scan for malware in production
- Store outside web root (not publicly accessible directly)
- Never execute uploaded files

Multer fields() for mixed uploads:

- `upload.fields([name, maxCount])`
- Handles multiple different file fields in one request
- Access via `req.files.fieldName`

Image validation beyond MIME type:

- Check magic bytes (file signature)
- Try to parse as image — invalid images will throw

```

1  const multer = require('multer')
2  const sharp = require('sharp')
3  const { z } = require('zod')
4
5  // Comprehensive file validation
6  const ALLOWED_MIMES = {
7    'image/jpeg': '.jpg',
8    'image/png': '.png',
9    'image/webp': '.webp',
10   'image/gif': '.gif',
11   'application/pdf': '.pdf',
12   'text/csv': '.csv'
13 }
14
15 const upload = multer({
16   storage: multer.memoryStorage(),
17   limits: { fileSize: 20 * 1024 * 1024, files: 10 },
18   fileFilter: (req, file, cb) => {
19     if (!ALLOWED_MIMES[file.mimetype]) {
20       return cb(new Error(`${file.mimetype} not supported`))
21     }
22     cb(null, true)
23   }
24 })
25
26 // Validate images are actually images (not disguised malware)
27 async function validateAndProcessImage(buffer, mimetype) {

```

```

28   try {

```

```

29  const image = sharp(buffer) // throws if not a valid image!
30  const metadata = await image.metadata()
31  if (!['jpeg', 'png', 'webp', 'gif'].includes(metadata.format)) {
32    throw new Error('Invalid image format')
33  }
34  if (metadata.width > 10000 || metadata.height > 10000) {
35    throw new Error('Image dimensions too large (max 10000x10000)')
36  }
37  // Strip metadata (EXIF can contain GPS, device info)
38  return image.withMetadata(false).toBuffer()
39 } catch(err) {
40   throw new Error(`Invalid image file: ${err.message}`)
41 }
42 }
43
44 // Product with multiple images
45 router.post('/products',
46   authenticate,
47   authorize('admin', 'editor'),
48   upload.fields([
49     { name: 'mainImage', maxCount: 1 },
50     { name: 'gallery', maxCount: 10 }
51   ]),
52   asyncHandler(async (req, res) => {
53     const images = []
54
55     if (req.files?.mainImage) {
56       const buf = await validateAndProcessImage(req.files.mainImage[0].buffer)
57       const url = await uploadToS3(buf, 'image/jpeg', 'products/main')
58       images.push({ type: 'main', url })
59     }
60
61     if (req.files?.gallery) {
62       for (const file of req.files.gallery) {
63         const buf = await validateAndProcessImage(file.buffer)
64         const url = await uploadToS3(buf, 'image/jpeg', 'products/gallery')
65         images.push({ type: 'gallery', url })
66       }
67     }
68
69     const product = await Product.create({ ...req.body, images })
70     res.status(201).json(product)
71   })
72 )

```

■ Interview Tip:

Always strip EXIF metadata from uploaded images (sharp's `withMetadata(false)`). Users uploading photos from phones include GPS coordinates, device info, and personal details in the EXIF data — which becomes public if you serve the original file.

What is graceful shutdown in Express?

■ Answer

Graceful shutdown stops accepting new requests, finishes existing ones, then exits. Critical for Kubernetes deployments and zero-downtime deploys.

Shutdown sequence:

1. Receive SIGTERM (Kubernetes, Docker stop, kill)
2. Set `isShuttingDown = true` → return 503 to new requests
3. `server.close()` — stop accepting new connections
4. Wait for in-flight requests to finish
5. Close database connections
6. Close cache connections
7. Exit with code 0

Force shutdown timeout:

- If shutdown takes too long → force exit
- Prevents stuck containers
- Kubernetes default grace period: 30 seconds

Zero downtime with Kubernetes:

- K8s sends SIGTERM, then waits `terminationGracePeriodSeconds`
- New traffic stops routing to pod
- Pod finishes existing requests and exits

```
1 const express = require('express')
2 const mongoose = require('mongoose')
3 const redis = require('ioredis').createClient()
4
5 const app = express()
6 let isShuttingDown = false
7
8 // Mark shutdown early so new requests return 503
9 app.use((req, res, next) => {
10   if (isShuttingDown) {
11     res.setHeader('Connection', 'close') // tell clients to reconnect
12     return res.status(503).json({ error: 'Server is shutting down' })
13   }
14   next()
15 })
16
17 // Your routes
18 app.use('/api/v1', require('./routes'))
19
20 // Start server
21 const server = app.listen(process.env.PORT || 3000, () => {
22   console.log('Server ready')
23 })
24
```

```
25 // Graceful shutdown handler
```

```

26 async function shutdown(signal) {
27   console.log(`${signal}: starting graceful shutdown`)
28   isShuttingDown = true
29
30   // Forced exit after 30s (match Kubernetes terminationGracePeriodSeconds)
31   const forceExit = setTimeout(() => {
32     console.error('Forced shutdown: timeout exceeded')
33     process.exit(1)
34   }, 30000).unref()
35
36   try {
37     // 1. Stop accepting new HTTP connections
38     await new Promise((resolve, reject) => server.close((err) => err ? reject(err) : resolve()))
39     console.log('HTTP server closed')
40
41     // 2. Close MongoDB
42     await mongoose.connection.close(false) // false = don't force close
43     console.log('MongoDB disconnected')
44
45     // 3. Close Redis
46     await redis.quit()
47     console.log('Redis disconnected')
48
49     clearTimeout(forceExit)
50     console.log('Graceful shutdown complete')
51     process.exit(0)
52
53   } catch (err) {
54     console.error('Shutdown error:', err)
55     process.exit(1)
56   }
57 }
58
59 process.on('SIGTERM', () => shutdown('SIGTERM')) // Kubernetes, Docker
60 process.on('SIGINT', () => shutdown('SIGINT')) // Ctrl+C
61
62 // Keep track of active connections for forceful drain if needed
63 let connections = 0
64 app.use((req, res, next) => { connections++; res.on('finish', () => connections--); next() })

```

■ Interview Tip:

Kubernetes sends SIGTERM and then waits terminationGracePeriodSeconds (default 30s) before sending SIGKILL. If your server doesn't exit within that window, Kubernetes kills it forcefully. Make sure your shutdown completes within 25-28 seconds to give buffer.

Q45 How to implement a health check endpoint?

■ Answer

Health check endpoints let load balancers and Kubernetes know if the service is ready.

Two types of health checks:

- /health (liveness) — is the process alive? Return 200 if app is running
- /ready (readiness) — can I receive traffic? Check all deps are connected

What to check in /ready:

- Database connection status
- Cache (Redis) connection status
- Required env variables set
- Any critical external service

What NOT to put in health checks:

- Complex DB queries — health check should be instant (<100ms)
- Business logic
- Rate limiting (health checks need to always succeed)

Kubernetes probe config:

- livenessProbe → /health — if fails, pod restarted
- readinessProbe → /ready — if fails, pod removed from load balancer

```
1 const mongoose = require('mongoose')
2 const redis = require('ioredis').createClient()
3 const os = require('os')
4
5 let startTime = Date.now()
6 let isReady = false
7
8 // Set ready after all connections established
9 mongoose.connect(MONGO_URI).then(() => {
10   isReady = true
11   console.log('Ready to accept traffic')
12 })
13
14 // /health - liveness probe (is the process alive?)
15 app.get('/health', (req, res) => {
16   // Simple check - if this responds, Node.js is running
17   res.json({
18     status: 'ok',
19     timestamp: new Date().toISOString(),
20     uptime: process.uptime(),
21     pid: process.pid
22   })
23 })
24
25 // /ready - readiness probe (can we serve traffic?)
```

```
26 app.get('/ready', async (req, res) => {
```

```

27  const checks = {
28      app: true,
29      database: false,
30      cache: false
31  }
32
33  // Check MongoDB
34  try {
35      await mongoose.connection.db.admin().ping()
36      checks.database = true
37  } catch { checks.database = false }
38
39  // Check Redis
40  try {
41      await redis.ping()
42      checks.cache = true
43  } catch { checks.cache = false }
44
45  const allReady = Object.values(checks).every(Boolean)
46
47  res.status(allReady ? 200 : 503).json({
48      status: allReady ? 'ready' : 'not ready',
49      checks,
50      timestamp: new Date().toISOString(),
51      uptime: process.uptime(),
52      version: process.env.npm_package_version,
53      memory: {
54          heapUsed: `${Math.round(process.memoryUsage().heapUsed/1024/1024)}MB`,
55          heapTotal: `${Math.round(process.memoryUsage().heapTotal/1024/1024)}MB`
56      }
57  })
58 })
59
60 // /metrics - Prometheus metrics (optional)
61 app.get('/metrics', (req, res) => {
62     res.setHeader('Content-Type', 'text/plain')
63     res.send([
64         `process_uptime_seconds ${process.uptime()}`,
65         `process_memory_heap_used_bytes ${process.memoryUsage().heapUsed}`,
66         `nodejs_version_info{version='${process.version}'} 1`
67     ].join('\n'))
68 })

```

■ Interview Tip:

Add these endpoints to EVERY Express service. Kubernetes needs /health and /ready to manage pod lifecycle correctly. Without them, Kubernetes can't distinguish between a starting pod and a crashed pod, causing unnecessary restarts or routing traffic to unhealthy pods.

What is API documentation with Swagger/OpenAPI in Express?

■ Answer

Swagger UI auto-generates interactive API documentation from your code.

Options:

- swagger-ui-express + swagger-jsdoc — JSDoc comments → Swagger spec
- tsoa — TypeScript decorators → routes + Swagger spec
- express-openapi — route-level validation + docs

swagger-jsdoc approach:

- Write JSDoc comments above routes
- swagger-jsdoc generates OpenAPI spec JSON
- swagger-ui-express serves interactive UI

Benefits:

- Frontend devs can explore API without reading code
- Auto-generated client SDKs
- API contract testing
- Request/response validation

Best practice:

- Only expose in development/staging
- Or protect with basic auth in production

```
1 const swaggerJsdoc = require('swagger-jsdoc')
2 const swaggerUi = require('swagger-ui-express')
3
4 const swaggerSpec = swaggerJsdoc({
5   definition: {
6     openapi: '3.0.0',
7     info: {
8       title: 'My API',
9       version: '1.0.0',
10      description: 'Full-Stack API by Kaushal Singh'
11    },
12    servers: [{ url: 'http://localhost:3000/api/v1' }],
13    components: {
14      securitySchemes: {
15        bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' }
16      }
17    },
18    security: [{ bearerAuth: [] }] // default security for all endpoints
19  },
20  apis: ['./routes/*.js', './models/*.js'] // scan for JSDoc comments
21 })
22
23 // Serve Swagger UI (dev/staging only!)
24 if (process.env.NODE_ENV !== 'production') {
```

```
25   app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec, {
```

```

26     explorer: true,
27     customCss: '.swagger-ui .topbar { display: none }'
28   )))
29   app.get('/api-docs.json', (req, res) => res.json(swaggerSpec))
30 }
31
32 // JSDoc in route file:
33 /**
34  * @swagger
35  * /users:
36  *   get:
37  *     summary: List all users
38  *     tags: [Users]
39  *     parameters:
40  *       - in: query
41  *         name: page
42  *         schema: { type: integer, default: 1 }
43  *     responses:
44  *       200:
45  *         description: List of users
46  *         content:
47  *           application/json:
48  *             schema:
49  *               type: object
50  *               properties:
51  *                 data: { type: array }
52  *                 pagination: { type: object }
53  */
54 router.get('/users', authenticate, ctrl.list)

```

■ Interview Tip:

Swagger UI saves massive time for frontend developers. They can explore your API, try out endpoints, and see request/response formats without asking you. It's also invaluable for onboarding new team members. Set it up in day one of every project.

Q47

What is the difference between PUT and PATCH in Express?

■ Answer

PUT and PATCH both update resources but with different semantics.

PUT — full replacement:

- Client sends COMPLETE representation of the resource
- Missing fields are set to null/default
- Idempotent — same request always same result
- Good for: replacing entire document

PATCH — partial update:

- Client sends only CHANGED fields
- Unspecified fields remain unchanged
- More efficient — less data sent
- Good for: updating one or two fields

When to use which:

- Updating user profile name only → PATCH
- Uploading new user profile (all fields) → PUT
- Most real-world updates → PATCH

MongoDB operations:

- PUT → findByIdAndReplace or set all fields
- PATCH → findByIdAndUpdate with \$set

```
1 // PUT — replace entire resource
2 router.put('/users/:id', authenticate, authorizeOwner, asyncHandler(async (req, res) => {
3   // Client sent ALL fields — we replace the entire document
4   const { name, email, age, bio, avatar } = req.body
5
6   // Validate that ALL required fields are present
7   if (!name || !email) {
8     return res.status(400).json({ error: 'PUT requires all required fields' })
9   }
10
11   const user = await User.findByIdAndReplace(
12     req.params.id,
13     { name, email, age, bio, avatar, updatedAt: new Date() },
14     { new: true, runValidators: true }
15   )
16   res.json(user)
17 }))
18
19 // PATCH — partial update
20 router.patch('/users/:id', authenticate, authorizeOwner, asyncHandler(async (req, res) => {
21   // Only update the provided fields
22   const allowedFields = ['name', 'bio', 'avatar', 'phone', 'preferences']
23
24   // Build update object — only allowed fields
```

```

25  const updates = {}
26  for (const [key, value] of Object.entries(req.body)) {
27    if (allowedFields.includes(key)) updates[key] = value
28  }
29
30  if (Object.keys(updates).length === 0) {
31    return res.status(400).json({ error: 'No valid fields to update' })
32  }
33
34  const user = await User.findByIdAndUpdate(
35    req.params.id,
36    { $set: updates }, // ONLY update provided fields
37    { new: true, runValidators: true }
38  )
39  res.json(user)
40  )))
41
42  // PATCH for specific actions
43  router.patch('/users/:id/password', authenticate, asyncHandler(async (req, res) => {
44    const { currentPassword, newPassword } = req.body
45    const user = await User.findById(req.params.id).select('+password')
46    if (!await bcrypt.compare(currentPassword, user.password))
47      return res.status(401).json({ error: 'Wrong current password' })
48    user.password = newPassword // Mongoose hook will hash it
49    await user.save()
50    res.json({ message: 'Password updated' })
51  }))
52
53  // PATCH for status changes
54  router.patch('/orders/:id/cancel', authenticate, asyncHandler(async (req, res) => {
55    const order = await Order.findOneAndUpdate(
56      { _id: req.params.id, userId: req.user.id, status: { $nin: ['shipped', 'delivered'] } },
57      { $set: { status: 'cancelled', cancelledAt: new Date(), cancelReason: req.body.reason } },
58      { new: true }
59    )
60    if (!order) return res.status(400).json({ error: 'Cannot cancel this order' })
61    res.json(order)
62  }))

```

■ Interview Tip:

PATCH is the right choice for 90% of update operations in real-world apps. PUT is technically correct for full replacements but rarely needed. Using PATCH also prevents accidentally clearing fields the user didn't intend to change.

How to implement an audit log in Express?

Answer

Audit logs record who did what, when — critical for compliance and debugging.

What to log:

- User ID (who)
- Action type (what: CREATE, UPDATE, DELETE)
- Resource type and ID (which resource)
- Before and after state (diff)
- IP address
- Timestamp
- Request ID (to correlate with HTTP logs)

Implementation approaches:

- Mongoose middleware — automatic on model operations
- Express middleware — manual logging in routes
- Event bus — decouple logging from routes

Compliance considerations:

- GDPR: audit logs may contain personal data
- Retention: how long to keep logs?
- Immutability: logs should not be modifiable

```
1  const AuditLog = require('./models/AuditLog')
2
3  // Audit log model
4  // const auditSchema = new Schema({
5  //   userId: { type: ObjectId, ref: 'User' },
6  //   action: { type: String, enum: ['CREATE', 'READ', 'UPDATE', 'DELETE'] },
7  //   resourceType: String, resourceId: String,
8  //   before: Mixed, after: Mixed,
9  //   ipAddress: String, userAgent: String,
10 //   requestId: String, timestamp: Date
11 // })
12
13 // Audit middleware factory
14 const audit = (action, resourceType, getResourceId) => {
15   return asyncHandler(async (req, res, next) => {
16     // Store original json to intercept response
17     const originalJson = res.json.bind(res)
18     res.json = async (data) => {
19       // Only log successful operations
20       if (res.statusCode < 400) {
21         await AuditLog.create({
22           userId: req.user?.id,
23           action,
24           resourceType,
25           resourceId: getResourceId ? getResourceId(req, data) : req.params.id,
26
27           after: action === 'DELETE' ? null : data,
```

```

27     ipAddress: req.ip,
28     userAgent: req.get('User-Agent'),
29     requestId: req.requestId,
30     timestamp: new Date()
31   }).catch(err => logger.error('Audit log failed:', err)) // don't fail on audit error
32 }
33   return originalJson(data)
34 }
35   next()
36 })
37 }
38
39 // Apply to routes
40 router.post('/users',
41   authenticate, authorize('admin'), validate(schema),
42   audit('CREATE', 'User', (req, res) => res.data?._id),
43   userCtrl.create
44 )
45
46 router.delete('/users/:id',
47   authenticate, authorize('admin'),
48   audit('DELETE', 'User', req => req.params.id),
49   userCtrl.delete
50 )
51
52 // Query audit logs
53 router.get('/admin/audit-logs', authenticate, authorize('admin'), asyncHandler(async (req, res) => {
54   const { userId, resourceType, action, startDate, endDate, page = 1 } = req.query
55   const filter = {}
56   if (userId) filter.userId = userId
57   if (resourceType) filter.resourceType = resourceType
58   if (action) filter.action = action
59   if (startDate || endDate) filter.timestamp = {}
60   if (startDate) filter.timestamp.$gte = new Date(startDate)
61   if (endDate) filter.timestamp.$lte = new Date(endDate)
62
63   const logs = await AuditLog.find(filter).sort({ timestamp: -1 })
64     .limit(50).skip((page-1)*50)
65     .populate('userId', 'name email')
66     .lean()
67   res.json({ logs })
68 })))

```

■ Interview Tip:

Use async audit logging that doesn't block the response. If the audit log write fails, the operation should still succeed — catch errors and log them separately. Never let audit logging failures fail the actual operation.

Q49 What is `express.static()`? How to serve static files?

■ Answer

`express.static()` is built-in middleware that serves static files from a directory.

Usage:

- `app.use(express.static('public'))` — serves files from `public/` folder
- `app.use('/assets', express.static('public'))` — serves at `/assets/` prefix
- `GET /style.css` → serves `public/style.css` automatically

Key options:

- `maxAge` — browser cache duration ('1d', '7d', '30d')
- `etag` — ETag conditional request support (default `true`)
- `index` — serve `index.html` for directories (default `'index.html'`)
- `dotfiles` — `'deny'` to block `.env`, `.htaccess`, hidden files

Production recommendation:

- Let Nginx serve static files (10x faster than Express)
- Or use CDN (CloudFront, Cloudflare) for global distribution
- Express static is perfectly fine for development

```
1  const path = require('path')
2
3  // Serve public/ directory
4  app.use(express.static(path.join(__dirname, 'public'), {
5    maxAge: '7d',           // browser caches for 7 days
6    etag: true,             // enables 304 Not Modified responses
7    dotfiles: 'deny'       // block .env, .git, .htaccess
8  }))
9
10 // Serve uploads at specific URL path
11 app.use('/uploads', express.static('uploads', { maxAge: '1d' }))
12 // GET /uploads/photo.jpg → serves uploads/photo.jpg
13
14 // Multiple static dirs (first match wins)
15 app.use(express.static('public'))
16 app.use(express.static('uploads'))
17
18 // SPA catch-all – serve React build
19 app.use(express.static(path.join(__dirname, 'dist')))
20 app.get('*', (req, res) => {
21   if (!req.path.startsWith('/api')) {
22     res.sendFile(path.join(__dirname, 'dist', 'index.html'))
23   }
24 })
```

■ Interview Tip:

In production, configure Nginx to serve your static files and only proxy `/api/*` to Node.js. Node.js never gets touched for static files — saving your event loop for actual API work.

Q50 How to implement request ID tracking in Express?

■ Answer

Request IDs link all log entries from a single HTTP request for easy debugging.

Why request IDs matter:

- One HTTP request generates dozens of log lines across services
- Without IDs: impossible to correlate related logs
- With IDs: filter logs by requestId to trace entire request flow
- Essential for microservices distributed tracing (Jaeger, Datadog)

Best practices:

- Check for incoming X-Request-Id header first (from gateway/client)
- Generate UUID if not present
- Echo back in response header for client debugging
- Include in every log entry automatically

```
1 const crypto = require('crypto')
2
3 // First middleware – assign request ID to every request
4 app.use((req, res, next) => {
5   req.id = req.headers['x-request-id'] || crypto.randomUUID()
6   res.setHeader('X-Request-Id', req.id) // echo back to client
7   next()
8 })
9
10 // Attach child logger with requestId context
11 app.use((req, res, next) => {
12   req.log = logger.child({ requestId: req.id, userId: req.user?.id })
13   next()
14 })
15
16 // All logs in route handlers automatically include requestId
17 app.get('/orders/:id', asyncHandler(async (req, res) => {
18   req.log.info({ orderId: req.params.id }, 'Fetching order')
19   const order = await Order.findById(req.params.id)
20   req.log.info({ status: order.status }, 'Order fetched')
21   // Both log lines share same requestId – easy to trace!
22   res.json(order)
23 })))
24
25 // Forward to downstream services (microservices)
26 const response = await axios.get('http://payment-service/status', {
27   headers: { 'X-Request-Id': req.id } // propagate the same ID
28 })
```

■ Interview Tip:

Use AsyncLocalStorage to propagate requestId through the entire async call chain without passing it as a function parameter. Set once in middleware, read anywhere — logger, DB helpers, email service, all automatically.

Q51 app.listen() vs http.createServer() — what is the difference?

Answer

Both start an HTTP server, but `http.createServer()` gives you more control.

app.listen(port):

- Convenience wrapper around `http.createServer(app).listen(port)`
- Returns the `http.Server` instance
- Simplest option for pure HTTP apps

http.createServer(app):

- REQUIRED when sharing one port with WebSocket (`ws`, `Socket.io`)
- REQUIRED for HTTPS (use `https.createServer` instead)
- Allows configuring server-level timeouts and options
- `app` acts as the 'request' event handler

Rule of thumb:

- Only HTTP, no WebSocket: `app.listen()` is fine
- Adding WebSocket: must use `http.createServer(app)`

```

1  const express = require('express')
2  const http = require('http')
3  const { WebSocketServer } = require('ws')
4
5  const app = express()
6  app.use(express.json())
7
8  // Simple: app.listen (HTTP only)
9  // const server = app.listen(3000)
10
11 // Manual: share one port for HTTP + WebSocket
12 const server = http.createServer(app)
13 const wss = new WebSocketServer({ server }) // same port!
14
15 wss.on('connection', (ws) => {
16   ws.send(JSON.stringify({ type: 'CONNECTED' }))
17 })
18
19 // Configure server-level timeouts (important for production!)
20 server.keepAliveTimeout = 65000 // must be > Nginx keepalive (60s)
21 server.headersTimeout = 66000 // slightly above keepAliveTimeout
22
23 server.listen(3000, () => console.log('HTTP + WS on :3000'))
24
25 // HTTPS:
26 const https = require('https')
27 const fs = require('fs')
28 const httpsServer = https.createServer({
29   key: fs.readFileSync('./ssl/key.pem'),

```

```

30   cert: fs.readFileSync('./ssl/cert.pem'),

```

```
31     minVersion: 'TLSv1.2'  
32   }, app)  
33   httpsServer.listen(443)
```

■ Interview Tip:

Set `server.keepAliveTimeout` to 65s when running behind Nginx. Nginx default keepalive is 60s — if Node.js closes connections at 5s (default), Nginx sends requests on closed connections and you get 502 errors under load.

How to implement idempotency keys in Express?

Answer

Idempotency keys prevent duplicate operations when clients retry failed requests.

The problem:

- User clicks Pay button twice rapidly
- Network failure causes client to retry request
- Both requests reach server → double charge!

Idempotency key solution:

- Client generates UUID once per operation
- Sends as Idempotency-Key: <uuid> header
- Server stores result in Redis keyed by this UUID
- Duplicate request → return cached result, no side effects

Key properties:

- Expire keys after 24 hours
- Scope keys by user ID (prevent key reuse across users)
- Use Redis NX to handle concurrent duplicates atomically

```
1 const redis = require('ioredis').createClient()
2
3 const idempotency = (ttl = 86400) => asyncHandler(async (req, res, next) => {
4   const key = req.headers['idempotency-key']
5   if (!key) return res.status(400).json({ error: 'Idempotency-Key header required' })
6
7   const cacheKey = `idem:${req.user.id}:${key}` // scoped per user!
8
9   // Check if already processed
10  const cached = await redis.get(cacheKey)
11  if (cached) {
12    const result = JSON.parse(cached)
13    res.setHeader('X-Idempotency-Replayed', 'true')
14    return res.status(result.status).json(result.body)
15  }
16
17  // Atomic lock – prevents concurrent duplicate requests
18  const locked = await redis.set(cacheKey, 'processing', 'EX', 30, 'NX')
19  if (!locked) return res.status(409).json({ error: 'Duplicate request in progress' })
20
21  // Intercept response to cache it
22  const origJson = res.json.bind(res)
23  res.json = async (body) => {
24    await redis.setex(cacheKey, ttl, JSON.stringify({ status: res.statusCode, body }))
25    return origJson(body)
26  }
27  next()
28 })
```

```
30 // Apply to payment and order endpoints
31 router.post('/payments', authenticate, idempotency(), paymentCtrl.create)
32 router.post('/orders', authenticate, idempotency(3600), orderCtrl.create)
33
34 // Client usage:
35 // const key = crypto.randomUUID() // generate ONCE, reuse on retry
36 // fetch('/payments', { headers: { 'Idempotency-Key': key }, ... })
```

■ Interview Tip:

The Redis NX (set if Not eXists) command is atomic — it prevents race conditions where two identical requests arrive simultaneously and both think they're the first. Only one gets the lock; the other gets a 409 Conflict.

Q53 How to implement soft delete pattern in Express?

■ Answer

Soft delete marks records as deleted instead of physically removing them from the database.

Why soft delete in production:

- Recover accidentally deleted records instantly
- Maintain audit trail — who deleted what and when
- Keep referential integrity — other records still reference the ID
- Compliance — GDPR, financial regulations often require retention

Mongoose implementation:

- Add deletedAt: Date field (null = active, timestamp = deleted)
- Pre-find middleware automatically excludes deleted records
- softDelete() instance method for consistency

Purge job:

- Periodically hard-delete records deleted > 90 days ago

```
1  const mongoose = require('mongoose')
2
3  const userSchema = new mongoose.Schema({
4    name: String, email: String,
5    deletedAt: { type: Date, default: null, index: true },
6    deletedBy: { type: mongoose.Types.ObjectId, ref: 'User', default: null }
7  }, { timestamps: true })
8
9  // Pre-find hook — auto-excludes deleted on ALL find operations
10 userSchema.pre(/^find/, function(next) {
11   this.where({ deletedAt: null }) // ← transparent to all queries
12   next()
13 })
14
15 // Soft delete method
16 userSchema.methods.softDelete = function(deletedById) {
17   this.deletedAt = new Date()
18   this.deletedBy = deletedById
19   return this.save()
20 }
21
22 const User = mongoose.model('User', userSchema)
23
24 // DELETE route — soft deletes
25 router.delete('/users/:id', authenticate, authorize('admin'),
26   asyncHandler(async (req, res) => {
27     const user = await User.findById(req.params.id) // pre-find skips deleted
28     if (!user) return res.status(404).json({ error: 'User not found' })
29     await user.softDelete(req.user.id)
30     res.json({ message: 'User deleted successfully' })
31   })
```

```

32 )
33
34 // Admin restore endpoint
35 router.post('/admin/users/:id/restore', authenticate, authorize('admin'),
36   asyncHandler(async (req, res) => {
37     // Must bypass the pre-find hook with direct MongoDB update
38     const result = await User.collection.updateOne(
39       { _id: new mongoose.Types.ObjectId(req.params.id) },
40       { $set: { deletedAt: null, deletedBy: null } }
41     )
42     if (!result.modifiedCount) return res.status(404).json({ error: 'User not found' })
43     res.json({ message: 'User restored successfully' })
44   })
45 )

```

■ Interview Tip:

The pre-find hook means you NEVER have to add { deletedAt: null } to your queries — it's applied globally and automatically. The only time you bypass it is for admin restore operations using direct collection access.

Q54 How to implement bulk operations in Express?

Answer

Bulk endpoints process many records in one request, far more efficient than N individual calls.

Common bulk operations:

- Bulk create — import CSV products, seed data
- Bulk update — apply 20% discount to all electronics
- Bulk delete — remove selected cart items
- Bulk upsert — sync data from external systems

Design considerations:

- Set a maximum batch size limit (e.g., 500 records)
- Validate ALL items before processing ANY
- Return per-item results for partial failures
- `ordered:false` in `insertMany` — don't stop on first error

Performance:

- `insertMany` is 10x faster than N individual inserts
- `updateMany` with `$in` is much faster than a loop

```
1 router.post('/products/bulk', authenticate, authorize('admin'),
2   asyncHandler(async (req, res) => {
3     const { products } = req.body
4
5     if (!Array.isArray(products) || products.length === 0)
6       return res.status(400).json({ error: 'products array is required' })
7     if (products.length > 500)
8       return res.status(400).json({ error: 'Maximum 500 products per batch' })
9
10    // Validate EVERY item before inserting ANY
11    const errors = []
12    const valid = []
13    products.forEach((p, index) => {
14      const result = productSchema.safeParse(p)
15      if (result.success) valid.push(result.data)
16      else errors.push({ index, issues: result.error.issues })
17    })
18    if (errors.length > 0)
19      return res.status(400).json({ error: 'Validation failed', errors })
20
21    // Insert in chunks of 100
22    let totalCreated = 0
23    for (let i = 0; i < valid.length; i += 100) {
24      const chunk = valid.slice(i, i + 100)
25      const result = await Product.insertMany(chunk, { ordered: false })
26      totalCreated += result.length
27    }
28    res.status(201).json({ created: totalCreated, message: `${totalCreated} products imported` })
29  })
```

```
30 )
31
32 // Bulk price update
33 router.patch('/products/bulk-discount', authenticate, authorize('admin'),
34   asyncHandler(async (req, res) => {
35     const { ids, discountPercent } = req.body
36     const result = await Product.updateMany(
37       { _id: { $in: ids } },
38       [{ $set: { price: { $multiply: ['$price', (100 - discountPercent) / 100] } } }]
39     )
40     res.json({ updated: result.modifiedCount })
41   })
42 )
```

■ **Interview Tip:**

insertMany with ordered:false is key for bulk imports — it continues inserting even when some documents fail (e.g., duplicate keys). With ordered:true (default), the first failure stops the entire batch, leaving you with partial data.

How to implement ETags for HTTP caching in Express?

■ Answer

ETags let clients cache responses and only re-download content when it actually changes.

How the ETag cycle works:

1. Server sends: ETag: 'abc123' with response
2. Client caches response + ETag value
3. Client next request: If-None-Match: 'abc123'
4. Server: still 'abc123' → 304 Not Modified (no body!)
5. Server: changed → 200 + new ETag + full response body

Express auto-ETags:

- Express generates weak ETags for `res.json()` by default
- `app.set('etag', 'strong')` — stronger MD5-based ETags
- `app.set('etag', false)` — disable entirely

304 vs 200:

- 304 has no response body — just headers
- 304 for a 50KB response = 200 bytes instead of 50KB

```

1  const crypto = require('crypto')
2
3  // Express auto-ETag (on by default — just enable strong mode)
4  app.set('etag', 'strong')
5  // res.json() automatically generates ETag header
6  // Browser sends If-None-Match on next request
7  // Express automatically returns 304 if ETag matches
8
9  // Manual ETag — control exactly when cache invalidates
10 router.get('/products', asyncHandler(async (req, res) => {
11   const products = await Product.find().lean()
12
13   // Hash the content to generate ETag
14   const etag = `${crypto.createHash('md5')
15     .update(JSON.stringify(products))
16     .digest('hex')}`
17
18   // Client has current version — skip body!
19   if (req.headers['if-none-match'] === etag) {
20     return res.status(304).end() // 304 = no body, saves bandwidth
21   }
22
23   res.setHeader('ETag', etag)
24   res.setHeader('Cache-Control', 'public, max-age=60, must-revalidate')
25   res.json(products)
26 })
27
28 // Simple ETag from last-modified timestamp
29 router.get('/products/:id', asyncHandler(async (req, res) => {

```

```

30 const product = await Product.findById(req.params.id)

```

```

31  const etag = `${product.updatedAt.getTime()}`
32  if (req.headers['if-none-match'] === etag) return res.status(304).end()
33  res.setHeader('ETag', etag)
34  res.setHeader('Cache-Control', 'public, max-age=300')
35  res.json(product)
36  )))
37
38  // Disable caching for private/sensitive data
39  router.get('/me', authenticate, (req, res) => {
40    res.setHeader('Cache-Control', 'private, no-store')
41    res.json(req.user) // never cached
42  })

```

■ Interview Tip:

ETags are most powerful for list endpoints. A 500-product JSON response (100KB) becomes a 200-byte 304 response for clients who already have it. At 10,000 requests/day, this saves significant bandwidth and improves perceived performance.

How to use Express with TypeScript?

Answer

TypeScript brings compile-time type safety to Express — catching bugs before they reach production.

Getting started:

- `npm install typescript @types/node @types/express ts-node-dev`
- `tsconfig.json`: `strict:true`, `target:ES2020`, `moduleResolution:node`
- Extend Express types in a `.d.ts` declaration file

Why extend Express types:

- `req.user` is not on `Express.Request` by default
- `req.id`, `req.log`, custom properties need declaration
- Without extending: TypeScript errors on `req.user`

Benefits:

- `req.body` is typed — catch missing fields at compile time
- Full autocomplete for all Express methods
- Safer refactoring — rename a field and TypeScript tells you everywhere to update

```

1 // src/types/express.d.ts – extend Express globally
2 declare global {
3   namespace Express {
4     interface Request {
5       id: string
6       user?: { id: string; role: 'user' | 'admin' | 'editor' }
7       log: import('pino').Logger
8     }
9   }
10 }
11
12 // src/controllers/users.ts
13 import { RequestHandler } from 'express'
14 import { z } from 'zod'
15
16 const createSchema = z.object({
17   name: z.string().min(2).max(50),
18   email: z.string().email(),
19   role: z.enum(['user', 'admin', 'editor']).default('user')
20 })
21 type CreateBody = z.infer<typeof createSchema> // type from schema!
22
23 // RequestHandler<Params, ResBody, ReqBody, Query>
24 export const createUser: RequestHandler<{}, any, CreateBody> =
25   async (req, res, next) => {
26     try {
27       const data = createSchema.parse(req.body) // data is CreateBody type
28       const user = await User.create(data) // fully typed!
29       res.status(201).json(user)
30     } catch (err) { next(err) }
  
```

```
31 }
32
33 export const getUser: RequestHandler<{ id: string }> =
34   async (req, res, next) => {
35     const { id } = req.params // TypeScript knows id: string
36     const user = await User.findById(id)
37     if (!user) return res.status(404).json({ error: 'Not found' })
38     res.json(user)
39   }
```

■ Interview Tip:

TypeScript + Zod is the ideal Express stack. Zod validates at runtime AND provides TypeScript types at compile time from the same schema. One source of truth eliminates the classic bug of validation schema and TypeScript interface getting out of sync.

Q57 What is express-validator vs Zod for validation?

■ Answer

Both validate request data but with very different philosophies and TypeScript support.

express-validator:

- Built as Express middleware from the ground up
- Long history, massive tutorial ecosystem
- No TypeScript type inference from schemas
- Validates req.body, req.params, req.query together

Zod:

- TypeScript-first — types derived automatically from schemas
- Runs in browser too — share frontend and backend validation
- Cleaner, more readable schema syntax
- `z.infer<typeof schema>` gives TypeScript type for free
- Recommended for all new projects in 2024+

Decision guide:

- New project with TypeScript: Zod without question
- Existing codebase using express-validator: keep it
- Frontend and backend sharing validation: Zod

```

1 // ■■ ZOD (recommended) ■■
2 const { z } = require('zod')
3
4 const registerSchema = z.object({
5   name: z.string().min(2).max(50).trim(),
6   email: z.string().email().toLowerCase(),
7   password: z.string().min(8)
8     .regex(/[A-Z]/, 'Need uppercase letter')
9     .regex(/[0-9]/, 'Need a number'),
10  age: z.number().int().min(18).optional()
11 })
12 // type RegisterData = z.infer<typeof registerSchema> // free TypeScript type!
13
14 const validate = (schema) => async (req, res, next) => {
15   try {
16     req.body = await schema.parseAsync(req.body)
17     next()
18   } catch(err) {
19     return res.status(400).json({
20       error: 'Validation failed',
21       details: err.errors?.map(e => ({ field: e.path.join('.'), message: e.message }))
22     })
23   }
24 }
25 router.post('/register', validate(registerSchema), ctrl.register)
26

```

```

27 // ■■ EXPRESS-VALIDATOR ■■

```

```
28 const { body, validationResult } = require('express-validator')
29 router.post('/register', [
30   body('name').trim().notEmpty().isLength({ min: 2, max: 50 }),
31   body('email').isEmail().normalizeEmail(),
32   body('password').isLength({ min: 8 }).matches(/[A-Z]/).matches(/[0-9]/),
33   (req, res, next) => {
34     const errors = validationResult(req)
35     if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() })
36     next()
37   }
38 ], ctrl.register)
```

■ Interview Tip:

Zod's killer feature: same schema validates req.body AND generates the TypeScript type. With express-validator you write validation rules AND a separate TypeScript interface — two places that can drift out of sync. Zod is one source of truth.

Q58 How to handle streaming responses in Express?

Answer

Streaming sends data to the client progressively instead of buffering everything first.

When to use streaming:

- Large file downloads (CSV exports, reports)
- Real-time data feeds (SSE)
- Large JSON collections (millions of records)
- Video/audio serving

Benefits:

- Client starts receiving data immediately
- Server uses constant memory (no buffering)
- Better for large datasets that would crash with `res.json()`

Stream types in Express:

- `fs.createReadStream` — file streaming
- Mongoose cursor — database streaming
- Transform streams — data transformation on the fly

```
1  const { Transform } = require('stream')
2
3  // Stream file directly to response
4  router.get('/files/:name', authenticate, (req, res) => {
5    const filePath = path.join('./uploads', path.basename(req.params.name))
6    const stream = require('fs').createReadStream(filePath)
7    stream.on('error', () => res.status(404).json({ error: 'File not found' }))
8    res.setHeader('Content-Type', 'application/octet-stream')
9    res.setHeader('Content-Disposition', `attachment; filename=${req.params.name}`)
10   stream.pipe(res) // user starts downloading immediately!
11 })
12
13 // Stream large CSV export – constant memory usage
14 router.get('/export/orders', authenticate, asyncHandler(async (req, res) => {
15   res.setHeader('Content-Type', 'text/csv')
16   res.setHeader('Content-Disposition', 'attachment; filename=orders.csv')
17
18   res.write('id,total,status,customer,date\n') // CSV header
19
20   const cursor = Order.find({ userId: req.user.id }).cursor()
21   for await (const order of cursor) {
22     res.write(`${order._id},${order.total},${order.status},${order.customer},${order.createdAt}\n`)
23   }
24   res.end() // done!
25   // Processes 1M orders using < 50MB memory
26 })
27
28 // Transform stream – process data on the fly
```

```
29 router.get('/export/json-to-csv', authenticate, (req, res) => {
```

```
30 res.setHeader('Content-Type', 'text/csv')
31 const jsonToCsv = new Transform({
32   objectMode: true,
33   transform(doc, enc, cb) { cb(null, `${doc.name},${doc.email}\n`) }
34 })
35 User.find().cursor().pipe(jsonToCsv).pipe(res)
36 })
```

■ Interview Tip:

The Mongoose cursor is the key pattern for streaming large database results. Instead of loading 1 million orders into memory with `find()`, the cursor fetches them in batches and you process them one-by-one. Memory stays flat regardless of data size.

Q59 How to prevent circular JSON errors in Express?

■ Answer

Circular references in objects cause `JSON.stringify` to throw `TypeError: cyclic object value`.

Common sources of circular references:

- Full Mongoose documents (not lean) — contain internal circular refs
- req or res objects passed into response accidentally
- Custom objects with parent → child → parent relationships

Solutions in order of preference:

- Use `.lean()` — returns plain JS objects (fastest fix)
- Call `.toJSON()` or `.toObject()` on Mongoose documents
- Define `toJSON` transform in Mongoose schema
- Use `flatted` or `safe-stable-stringify` packages

Best practice:

- Always `.lean()` for read-only queries — no circular refs AND 2-3x faster

```
1  const mongoose = require('mongoose')
2
3  // Problem: full Mongoose document may have circular refs
4  const user = await User.findById(id) // Mongoose Document object
5  // res.json(user) ← risky, includes Mongoose internals
6
7  // ■ Solution 1: lean() — plain JS object, zero overhead
8  const userSafe = await User.findById(id).lean()
9  res.json(userSafe) // clean, fast, no circular refs
10
11 // ■ Solution 2: .toJSON() — Mongoose converts safely
12 const userDoc = await User.findById(id)
13 res.json(userDoc.toJSON()) // or .toObject()
14
15 // ■ Solution 3: Schema toJSON transform (best — automatic everywhere)
16 const userSchema = new mongoose.Schema({ name: String, password: String }, {
17   toJSON: {
18     virtuals: true,
19     versionKey: false, // remove __v field
20     transform: (doc, ret) => {
21       ret.id = ret._id // rename _id to id
22       delete ret._id
23       delete ret.password // ALWAYS hide password
24       return ret
25     }
26   }
27 })
28
29 // ■ Solution 4: select only needed fields
30 const users = await User.find().select('name email role -_id').lean()
```

```
32 // For debugging circular objects in logs (not for HTTP response):  
33 const flattened = require('flatted')  
34 console.log(flatted.stringify(circularObject)) // safe circular-aware stringify
```

■ Interview Tip:

The toJSON transform in the Mongoose schema is the most elegant solution — define it once, and every `res.json(user)` automatically removes sensitive fields and formats correctly. No need to manually clean up in every route handler.

How to implement content negotiation in Express?

Answer

Content negotiation serves different data formats from the same endpoint based on the Accept header.

Accept header values clients send:

- Accept: application/json — API client wants JSON
- Accept: text/html — browser wants HTML page
- Accept: text/csv — data export request
- Accept: */* — accept any format (default)

Express `res.format()`:

- Reads the Accept header automatically
- Calls the matching format handler
- Sends 406 Not Acceptable if no match (unless default provided)

Practical use cases:

- Admin dashboard: same `/users` route serves HTML to browser, JSON to API
- Reports: same endpoint returns JSON or CSV based on Accept header

```
1 // Same endpoint serves HTML or JSON based on Accept header
2 router.get('/users', authenticate, asyncHandler(async (req, res) => {
3   const users = await User.find().lean()
4
5   res.format({
6     'application/json': () => {
7       res.json(users) // API clients
8     },
9     'text/html': () => {
10      res.render('users/list', { users }) // browser
11    },
12     'text/csv': () => {
13      const csv = ['name,email,role',
14        ...users.map(u => `${u.name},${u.email},${u.role}`)
15      ].join('\n')
16      res.setHeader('Content-Disposition', 'attachment; filename=users.csv')
17      res.type('csv').send(csv) // CSV download
18    },
19     default: () => {
20      res.status(406).json({ error: 'Not Acceptable', supported: ['json','html','csv'] })
21    }
22  })
23 })
24
25 // Manual check with req.accepts()
26 router.get('/report', authenticate, asyncHandler(async (req, res) => {
27   const format = req.accepts(['json', 'csv', 'pdf'])
28   if (!format) return res.status(406).json({ error: 'Unsupported format' })
29   const data = await buildReport()
30   if (format === 'json') return res.json(data)
```

```
31   if (format === 'csv') return res.type('csv').send(dataToCsv(data))
32   const pdfPath = await dataToPdf(data)
33   res.download(pdfPath, 'report.pdf')
34 }}
```

■ **Interview Tip:**

Content negotiation with `res.format()` eliminates duplicate endpoints. Instead of `/api/users` (JSON) and `/users` (HTML), one route serves both. Particularly useful for admin dashboards that need a UI for humans and an API for automation.

Q61 How to implement database transactions in Express routes?

■ Answer

Transactions guarantee that multiple database operations succeed or fail together atomically.

When you need transactions:

- Money transfer: debit account A + credit account B must be atomic
- Place order: create order + reduce inventory + create payment record
- Registration: create user + create profile + send welcome email log

MongoDB transactions require:

- MongoDB 4.0+ with Replica Set (even single-node dev needs --replSet)
- `session.withTransaction()` handles commit, rollback, and transient error retry
- EVERY operation inside must receive the session option

Forget session → operation runs OUTSIDE the transaction!

- Silent bug: some ops in transaction, some not

```

1  const mongoose = require('mongoose')
2
3  router.post('/transfer', authenticate, asyncHandler(async (req, res) => {
4    const { fromId, toId, amount } = req.body
5    const session = await mongoose.startSession()
6
7    try {
8      let result
9      await session.withTransaction(async () => {
10        // 1. Debit sender – atomic check + decrement
11        const sender = await Account.findOneAndUpdate(
12          { _id: fromId, userId: req.user.id, balance: { $gte: amount } },
13          { $inc: { balance: -amount } },
14          { session, new: true } // ← session REQUIRED
15        )
16        if (!sender) throw new Error('Insufficient funds or account not found')
17
18        // 2. Credit receiver
19        await Account.findByIdAndUpdate(
20          toId, { $inc: { balance: amount } }, { session } // ← session REQUIRED
21        )
22
23        // 3. Audit log
24        const [tx] = await Transaction.create([
25          { from: fromId, to: toId, amount, status: 'completed' }
26        ], { session }) // ← array + session for create with session
27
28        result = tx
29      }) // auto-commit on success, auto-rollback on any throw
30
31      res.json({ success: true, transactionId: result._id })
32    } finally {

```

```

33    session.endSession() // ← ALWAYS end session!

```

```

34 }
35 )))
36
37 // PostgreSQL transactions (pg pool)
38 const client = await pgPool.connect()
39 try {
40   await client.query('BEGIN')
41   await client.query('UPDATE accounts SET balance = balance - $1 WHERE id = $2', [amount, fromId])
42   await client.query('UPDATE accounts SET balance = balance + $1 WHERE id = $2', [amount, toId])
43   await client.query('COMMIT')
44 } catch(err) { await client.query('ROLLBACK'); throw err }
45 finally { client.release() } // ← ALWAYS release!

```

■ Interview Tip:

withTransaction() automatically retries on transient write conflicts (a common occurrence in distributed MongoDB). Manual startTransaction/commitTransaction won't retry — you'd need to implement retry logic yourself. Always prefer withTransaction().

How to implement API key authentication in Express?

Answer

API keys authenticate server-to-server calls without user sessions or JWTs.

API keys vs JWT:

- API keys: static, long-lived, for machines/integrations
- JWT: dynamic, short-lived, for user sessions
- API keys: simpler to use (no expiry, no refresh)
- API keys: harder to revoke (must store in DB)

Security best practices:

- Store HASH of API key in database (SHA-256 is fine)
- Never log or expose the plaintext key after creation
- Scope keys: read-only, write, admin permissions
- Rate limit per API key
- Track last-used timestamp for audit

```
1  const crypto = require('crypto')
2
3  // Generate API key pair: return plain to user, store hash
4  function createApiKey() {
5    const prefix = process.env.NODE_ENV === 'production' ? 'sk_live' : 'sk_test'
6    const key = `${prefix}_${crypto.randomBytes(32).toString('base64url')}`
7    const hash = crypto.createHash('sha256').update(key).digest('hex')
8    return { key, hash }
9  }
10
11 // Create key endpoint
12 router.post('/api-keys', authenticate, asyncHandler(async (req, res) => {
13   const { name, scopes = ['read']} = req.body
14   const { key, hash } = createApiKey()
15   await ApiKey.create({ userId: req.user.id, name, hash, scopes })
16   res.status(201).json({
17     key, // ← show ONCE only!
18     warning: 'Save this key – it cannot be retrieved again'
19   })
20 }))
21
22 // API key authentication middleware
23 const apiKeyAuth = asyncHandler(async (req, res, next) => {
24   const raw = req.headers['x-api-key'] ||
25     req.headers.authorization?.replace('Bearer ', '')
26   if (!raw) return res.status(401).json({ error: 'API key required' })
27
28   const hash = crypto.createHash('sha256').update(raw).digest('hex')
29   const apiKey = await ApiKey.findOne({ hash, active: true })
30   if (!apiKey) return res.status(401).json({ error: 'Invalid API key' })
31
32   // Async update last-used (don't await – don't slow down request)
```

```
33   apiKey.updateOne({ _id: apiKey._id }, { $set: { lastUsedAt: new Date() } }).catch(()=>{})
34
35   req.user = { id: apiKey.userId, scopes: apiKey.scopes }
36   next()
37 })
38
39 // Apply to machine-to-machine routes
40 app.use('/api/v1/webhooks', apiKeyAuth)
41 app.use('/api/v1/integrations', apiKeyAuth)
```

■ Interview Tip:

Hash API keys with SHA-256 (fast hash is fine here, unlike passwords). API keys are long and random — impossible to brute-force even with fast hashing. The reason you don't store plaintext: if your DB is breached, all API keys are immediately invalidated.

Q63

How to implement circuit breaker pattern in Express?

■ Answer

Circuit breaker prevents your service from continuously hitting a failing external dependency.

Without circuit breaker:

- Payment API is down
- Every request waits for 30s timeout
- 1000 req/min × 30s = 30,000 hanging requests
- Your server runs out of connections and crashes

With circuit breaker (opossum library):

- After 50% failures: circuit OPENS
- New requests fail immediately (fast fail)
- After 30s: circuit goes HALF-OPEN (try one request)
- If success: CLOSSES again
- Fallback function returns cached/default data

```

1  const CircuitBreaker = require('opossum')
2
3  async function callPaymentAPI(orderId) {
4      const res = await fetch(`https://payment.api.com/orders/${orderId}`, { signal: AbortSignal.timeout(30000) })
5      if (!res.ok) throw new Error(`Payment API error: ${res.status}`)
6      return res.json()
7  }
8
9  const paymentBreaker = new CircuitBreaker(callPaymentAPI, {
10     timeout: 3000,           // fail if > 3 seconds
11     errorThresholdPercentage: 50, // open if 50% of last 10 calls fail
12     resetTimeout: 30000,      // try again after 30 seconds
13     volumeThreshold: 5        // at least 5 calls before circuit can open
14 })
15
16 // Fallback – return default data when circuit is OPEN
17 paymentBreaker.fallback((orderId) => ({
18     orderId, status: 'unknown',
19     message: 'Payment service temporarily unavailable. Please retry later.',
20     cached: true
21 })))
22
23 // Monitor state changes – alert your team!
24 paymentBreaker.on('open', () => {
25     logger.error('Payment circuit OPEN – failing fast!')
26     // alertSlack('#incidents', 'Payment API circuit breaker opened!')
27 })
28 paymentBreaker.on('close', () => logger.info('Payment circuit closed – recovered'))
29 paymentBreaker.on('halfOpen', () => logger.info('Payment circuit half-open – testing'))
30
31 router.get('/orders/:id/payment', authenticate, asyncHandler(async (req, res) => {
32     const status = await paymentBreaker.fire(req.params.id) // circuit protected
  
```

```
33     res.json(status) // may be fallback data if circuit open
34   )))
```

■ Interview Tip:

The circuit breaker pattern is one of the most important reliability patterns for microservices. Without it, one slow dependency can cascade into your entire service being down. With it, failures are isolated and users get graceful degradation.

How to implement background job queuing in Express?

■ Answer

Background job queues process heavy operations asynchronously without blocking HTTP responses.

Why queue background jobs:

- PDF generation takes 5 seconds — don't block HTTP for that
- Email sending can fail — queue with automatic retry
- Image processing, video transcoding — CPU intensive
- Report generation — DB heavy

BullMQ (Redis-backed):

- Jobs survive server restarts
- Multiple workers can process in parallel
- Built-in retry with backoff
- Job progress tracking

Pattern: 202 Accepted + job polling

- Return jobId immediately with 202
- Client polls GET /jobs/:id for status/result

```

1  const { Queue, Worker } = require('bullmq')
2  const redisConfig = { host: process.env.REDIS_HOST, port: 6379 }
3
4  // Create queue
5  const emailQueue = new Queue('emails', { connection: redisConfig })
6  const reportQueue = new Queue('reports', { connection: redisConfig })
7
8  // Route: add job, return immediately
9  router.post('/reports', authenticate, asyncHandler(async (req, res) => {
10     const job = await reportQueue.add('generate', {
11       userId: req.user.id,
12       type: req.body.type,
13       filters: req.body.filters
14     }, {
15       attempts: 3,
16       backoff: { type: 'exponential', delay: 2000 }
17     })
18     res.status(202).json({ jobId: job.id, status: 'queued' }) // 202 Accepted
19     // Client polls GET /reports/status/:jobId
20   })))
21
22 // Polling endpoint
23 router.get('/reports/status/:jobId', authenticate, asyncHandler(async (req, res) => {
24     const job = await reportQueue.getJob(req.params.jobId)
25     if (!job) return res.status(404).json({ error: 'Job not found' })
26     const state = await job.getState() // 'waiting', 'active', 'completed', 'failed'
27     res.json({ jobId: job.id, state, result: job.returnvalue, failedReason: job.failedReason })
28   })))

```

```
30 // Worker (separate process or same server)
31 const reportWorker = new Worker('reports', async (job) => {
32   const { userId, type, filters } = job.data
33   await job.updateProgress(10) // track progress
34   const data = await fetchReportData(filters)
35   await job.updateProgress(50)
36   const url = await generateAndUploadPdf(data)
37   await job.updateProgress(100)
38   return { url, generatedAt: new Date() } // returnvalue
39 }, { connection: redisConfig, concurrency: 3 })
```

■ Interview Tip:

The 202 Accepted + polling pattern is the correct REST approach for long-running operations. Never hold a client waiting more than a few seconds. Return a jobId, let them poll. For real-time updates, push job completion via WebSocket or SSE.

Q65 How to implement a reverse proxy with Express?

■ Answer

Express can act as an API gateway, routing requests to different backend services.

API gateway use cases:

- Route /api/users to user-service
- Route /api/orders to order-service
- Add authentication before proxying
- Rate limit at gateway level
- Transform requests/responses

http-proxy-middleware package:

- Most popular Express proxy middleware
- Supports WebSocket proxying
- Path rewriting, target routing
- Request/response hooks

What to handle at gateway:

- Authentication — verify JWT before proxying
- Rate limiting — protect all downstream services
- Request ID propagation — for distributed tracing

```
1  const { createProxyMiddleware } = require('http-proxy-middleware')
2
3  const SERVICES = {
4    users:    process.env.USERS_SERVICE    || 'http://user-svc:3001',
5    orders:   process.env.ORDERS_SERVICE   || 'http://order-svc:3002',
6    products: process.env.PRODUCTS_SERVICE || 'http://product-svc:3003'
7  }
8
9  // Global auth for all proxied routes
10 app.use('/api/', authenticate) // verify JWT before ANY proxying
11 app.use('/api/', rateLimit({ windowMs: 60000, max: 100 }))
12
13 // Proxy factory
14 const proxy = (target, pathPrefix) => createProxyMiddleware({
15   target,
16   changeOrigin: true,
17   pathRewrite: { [`^/api${pathPrefix}`]: '' }, // strip prefix
18   on: {
19     proxyReq: (proxyReq, req) => {
20       // Inject context for downstream services
21       proxyReq.setHeader('X-User-Id', req.user.id)
22       proxyReq.setHeader('X-User-Role', req.user.role)
23       proxyReq.setHeader('X-Request-Id', req.id)
24     },
25     error: (err, req, res) => {
26       logger.error({ err, service: target }, 'Proxy error')
27
28       res.status(502).json({ error: 'Service temporarily unavailable' })
29     }
30   }
31 })
```

```
28     }  
29   }  
30 })  
31  
32 // Mount proxies  
33 app.use('/api/users', proxy(SERVICES.users, '/users'))  
34 app.use('/api/orders', proxy(SERVICES.orders, '/orders'))  
35 app.use('/api/products', proxy(SERVICES.products, '/products'))
```

■ Interview Tip:

The API gateway pattern centralizes cross-cutting concerns (auth, rate limiting, logging, tracing) in one place. Downstream microservices become simpler — they trust that the gateway has already authenticated the request and just process the business logic.

How to implement request retry with exponential backoff?

Answer

Automatically retry failed external API calls with increasing delays between attempts.

When to retry:

- 5xx errors (502, 503, 504) — server-side issues, usually transient
- Network errors (ECONNRESET, ETIMEDOUT, ECONNREFUSED)
- DO NOT retry 4xx — those are client errors, retrying won't help

Exponential backoff formula:

- $\text{delay} = \text{baseDelay} \times 2^{\text{attempt}} + \text{random jitter}$
- Attempt 0: 1s, Attempt 1: 2s, Attempt 2: 4s, etc.
- Jitter prevents thundering herd after an outage

Libraries:

- axios-retry — easiest for axios
- p-retry — general-purpose promise retry
- Manual implementation for full control

```

1  const axiosRetry = require('axios-retry')
2  const axios = require('axios')
3
4  // axios-retry — simplest approach
5  axiosRetry(axios, {
6    retries: 3,
7    retryDelay: axiosRetry.exponentialDelay, // 1s, 2s, 4s
8    retryCondition: (err) =>
9      axiosRetry.isNetworkOrIdempotentRequestError(err) || err.response?.status >= 500
10 })
11
12 // Manual retry with exponential backoff + jitter
13 async function fetchWithRetry(url, options = {}, maxAttempts = 3) {
14   for (let attempt = 0; attempt < maxAttempts; attempt++) {
15     try {
16       const res = await axios({ url, timeout: 5000, ...options })
17       return res.data
18     } catch(err) {
19       const status = err.response?.status
20       const isLastAttempt = attempt === maxAttempts - 1
21       const shouldRetry = !status || status >= 500 // retry network errors and 5xx
22
23       if (isLastAttempt || !shouldRetry) throw err
24
25       // Exponential backoff: 1000ms, 2000ms, 4000ms + random jitter
26       const delay = Math.min(1000 * Math.pow(2, attempt) + Math.random() * 200, 10000)
27       logger.warn({ attempt, delay, url }, 'Retrying failed request')
28       await new Promise(r => setTimeout(r, delay))
29     }
30   }

```

```
31 }
32
33 // Usage in Express route
34 router.get('/weather/:city', asyncHandler(async (req, res) => {
35   const data = await fetchWithRetry(
36     `https://api.weather.com/v1/${req.params.city}`,
37     { headers: { 'Authorization': `Bearer ${process.env.WEATHER_KEY}` } },
38     3
39   )
40   res.json(data)
41 })))
```

■ Interview Tip:

Always add jitter (random milliseconds) to retry delays. Without jitter, if 1000 clients all fail simultaneously and all retry at exactly 2 seconds, you hit the recovering server with 1000 requests at once — potentially crashing it again. Jitter spreads them out.

Q67 How to generate and serve Swagger/OpenAPI docs in Express?

■ Answer

Swagger UI provides interactive API documentation directly from your Express code.

swagger-jsdoc approach (code-first):

- Write JSDoc comments above route handlers
- swagger-jsdoc generates OpenAPI spec JSON
- swagger-ui-express serves interactive Swagger UI

Benefits:

- Frontend devs explore your API without reading code
- Auto-generate client SDKs (openapi-generator)
- Try endpoints directly in the browser
- Acts as living documentation that stays in sync

Security note:

- Only expose Swagger UI in dev/staging
- Or protect with basic auth in production
- Your API schema reveals your entire surface area

```
1  const swaggerJsdoc = require('swagger-jsdoc')
2  const swaggerUi = require('swagger-ui-express')
3
4  const spec = swaggerJsdoc({
5    definition: {
6      openapi: '3.0.0',
7      info: { title: 'My API', version: '1.0.0' },
8      servers: [{ url: '/api/v1' }],
9      components: {
10       securitySchemes: {
11         bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' }
12       }
13     },
14     security: [{ bearerAuth: [] }]
15   },
16   apis: ['./routes/*.js'] // scan JSDoc comments
17 })
18
19 if (process.env.NODE_ENV !== 'production') {
20   app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(spec, { explorer: true }))
21   app.get('/api-docs.json', (req, res) => res.json(spec))
22 }
23
24 // JSDoc example in routes/users.js
25 /**
26  * @swagger
27  * /users:
28  *   get:
```

```
29  * summary: List all users
```

```
30 *      tags: [Users]
31 *      parameters:
32 *        - in: query
33 *          name: page
34 *          schema: { type: integer, default: 1 }
35 *      responses:
36 *        200:
37 *          description: List of users
38 *          content:
39 *            application/json:
40 *              schema:
41 *                type: object
42 *                properties:
43 *                  data: { type: array }
44 *                  pagination: { type: object }
45 */
46 router.get('/users', authenticate, ctrl.list)
```

■ Interview Tip:

Swagger UI saves hours of onboarding time for new team members and frontend developers. Instead of reading code or asking questions about request/response formats, they explore the interactive docs and try requests directly. Set it up on day one.

How to handle webhook events in Express?

Answer

Webhooks are HTTP callbacks from external services when events occur in their system.

Webhook security (critical!):

- Provider signs payload with HMAC-SHA256 using shared secret
- You verify signature — reject if invalid
- Must use raw body Buffer for signature check — parse JSON AFTER verification
- Respond with 200 within 10-30 seconds or provider retries

Idempotency for webhooks:

- Providers retry failed webhooks
- Store processed event IDs — check before processing
- Processing same event twice must be safe

Process asynchronously:

- Queue the event, acknowledge immediately with 200
- Heavy processing in background worker

```
1 const crypto = require('crypto')
2
3 // ■ Must use express.raw() – raw body needed for signature
4 router.post('/webhooks/stripe',
5   express.raw({ type: 'application/json' }), // NOT express.json()!
6   asyncHandler(async (req, res) => {
7     const sig = req.headers['stripe-signature']
8
9     let event
10    try {
11      event = stripe.webhooks.constructEvent(req.body, sig, process.env.STRIPE_WEBHOOK_SECRET)
12    } catch(err) {
13      logger.warn('Invalid Stripe webhook signature')
14      return res.status(400).json({ error: 'Invalid signature' })
15    }
16
17    // Idempotency – skip already-processed events
18    const exists = await WebhookEvent.exists({ stripeEventId: event.id })
19    if (exists) return res.json({ received: true, duplicate: true })
20
21    // Queue for async processing – respond 200 immediately!
22    await webhookQueue.add('stripe', { event })
23    await WebhookEvent.create({ stripeEventId: event.id, type: event.type })
24    res.json({ received: true })
25  })
26 )
27
28 // Generic signature verification helper
29 function verifyHmac(payload, signature, secret) {
```

```
30 const expected = crypto.createHmac('sha256', secret).update(payload).digest('hex')
```

```
31   try {  
32     return crypto.timingSafeEqual(Buffer.from(signature, 'hex'), Buffer.from(expected, 'hex'))  
33   } catch { return false }  
34 }
```

■ **Interview Tip:**

The `express.raw()` middleware is essential for webhook routes. If you parse the body with `express.json()` first, the raw bytes Buffer is gone. Stripe computes the HMAC over the raw bytes — re-serializing parsed JSON can produce different bytes, failing verification.

Q69 What are Express performance best practices?

■ Answer

Performance optimization in Express requires measuring first, then applying targeted fixes.

Top 5 highest-impact optimizations:

- 1. Use `.lean()` for read-only Mongoose queries (2-3x faster)
- 2. Enable gzip compression (70-80% response size reduction)
- 3. Add Redis caching for expensive DB queries
- 4. Run parallel async ops with `Promise.all` (not sequential `await`)
- 5. Add database indexes for queried fields

Quick wins list:

- Set `NODE_ENV=production` (Express enables optimizations)
- Use PM2 cluster mode (use all CPU cores)
- Increase `UV_THREADPOOL_SIZE` for bcrypt-heavy auth
- Limit payload size: `express.json({ limit: '100kb' })`
- Stream large responses — never `res.json(1M records)`

```

1 // 1. lean() – biggest easy win for read endpoints
2 const users = await User.find({ active: true }).lean() // 2-3x faster
3 const products = await Product.find().select('name price -_id').lean() // + projection
4
5 // 2. Compression
6 app.use(require('compression')()) // 70-80% response size reduction
7
8 // 3. Parallel ops – don't await sequentially!
9 // ■ Sequential: 100ms + 200ms + 150ms = 450ms
10 const user = await User.findById(id).lean()
11 const orders = await Order.find({ userId: id }).lean()
12 const stats = await getStats(id)
13
14 // ■ Parallel: max(100, 200, 150) = 200ms
15 const [user2, orders2, stats2] = await Promise.all([
16   User.findById(id).lean(),
17   Order.find({ userId: id }).lean(),
18   getStats(id)
19 ])
20
21 // 4. Cache expensive queries
22 const cacheKey = `dashboard:${userId}`
23 const cached = await redis.get(cacheKey)
24 if (cached) return res.json(JSON.parse(cached))
25 const data = await buildDashboard(userId) // expensive
26 await redis.setex(cacheKey, 300, JSON.stringify(data))
27 res.json(data)
28
29 // 5. Load test to find bottlenecks
30 // npx autocannon -c 100 -d 30 http://localhost:3000/api/products

```

```

31 // Shows: req/sec, latency p50/p95/p99, errors

```



```
32
33 // 6. Profile with clinic.js
34 // npx clinic doctor -- node app.js
35 // npx clinic flame -- node app.js
```

■ **Interview Tip:**

Run autocannon load test BEFORE and AFTER optimizing. Without measurement, you don't know if your optimization actually helped. A simple test: `autocannon -c 100 -d 30 localhost:3000/api/products`. Compare req/sec and p99 latency before and after each change.

What is Express 5? Key changes from Express 4?

Answer

Express 5 (currently in RC) brings async error handling and updated route syntax.

Most important change: async errors caught automatically

- Express 4: async errors in route handlers are silently dropped
- Express 5: async errors automatically passed to next(err)
- No more asyncHandler() wrapper needed
- No more .catch(next) at the end of every chain

Route path changes:

- Optional segment: :id? → {/:id} in Express 5
- Named wildcard: * → *name in Express 5
- Named capture group: :id(d+) still works

Other removals:

- app.del() removed — use app.delete()
- res.json(body, status) removed — use res.status(code).json(body)

```

1 // Express 4 – asyncHandler REQUIRED for async routes
2 const asyncHandler = fn => (req, res, next) =>
3   Promise.resolve(fn(req, res, next)).catch(next)
4
5 app.get('/users/:id', asyncHandler(async (req, res) => {
6   const user = await User.findById(req.params.id) // throws if DB error
7   if (!user) throw new NotFoundError('User')
8   res.json(user)
9 }))
10
11 // ■ Express 5 – async errors caught natively!
12 app.get('/users/:id', async (req, res) => {
13   const user = await User.findById(req.params.id) // throws → auto next(err)!
14   if (!user) throw new NotFoundError('User') // throw → auto next(err)!
15   res.json(user)
16   // No wrapper needed. Error handler still catches everything.
17 })
18
19 // Express 5 path syntax
20 // Express 4: app.get('/users/:id?', handler)
21 // Express 5: app.get('/users{/:id}', handler) // optional segment
22
23 // Express 4: app.get('/files/*', handler)
24 // Express 5: app.get('/files/*filepath', handler) // must name wildcard
25
26 // Install Express 5 RC
27 // npm install express@latest // 5.x when stable
28 // npm install express@next // RC/beta
29

```

```

30 // Error handler unchanged – still 4 params required

```

```
31 app.use((err, req, res, next) => {  
32   res.status(err.statusCode || 500).json({ error: err.message })  
33 })
```

■ Interview Tip:

Express 5's automatic async error handling is the single most impactful change for developers. It eliminates the `asyncHandler` boilerplate that every Express 4 project requires. Many projects will upgrade for this alone.

Q71 How to implement HTTPS redirect in Express?

■ Answer

Redirecting HTTP to HTTPS ensures all traffic is encrypted in production.

Why HTTPS redirect:

- HTTP traffic is plaintext — credentials visible to anyone on network
- Google penalizes non-HTTPS sites in search rankings
- Modern browsers mark HTTP sites as 'Not Secure'
- Required for cookies with Secure flag to work

Express approach:

- Check req.secure or X-Forwarded-Proto header
- Redirect to https:// with 301 (permanent)
- Requires app.set('trust proxy', 1) behind Nginx

Better: handle in Nginx before Express

- Nginx redirect = request never hits Node.js
- Faster, uses no Node.js resources at all

```

1 // Express HTTPS redirect middleware
2 app.set('trust proxy', 1) // MUST be set — trust Nginx X-Forwarded-Proto
3
4 app.use((req, res, next) => {
5   const isHttps = req.secure || req.headers['x-forwarded-proto'] === 'https'
6
7   if (!isHttps && process.env.NODE_ENV === 'production') {
8     const redirectUrl = `https://${req.hostname}${req.originalUrl}`
9     return res.redirect(301, redirectUrl) // 301 = permanent (browser caches)
10  }
11  next()
12 })
13
14 // Add HSTS — after redirect, browser auto-uses HTTPS for 1 year
15 app.use(require('helmet')({
16   hsts: {
17     maxAge: 31536000, // 1 year in seconds
18     includeSubDomains: true,
19     preload: true
20   }
21 })))
22
23 // ■■ BETTER: Nginx handles redirect (no Node.js involved) ■■
24 // nginx.conf:
25 // server {
26 //   listen 80;
27 //   server_name myapp.com www.myapp.com;
28 //   return 301 https://$host$request_uri; // redirect before Node.js
29 // }
```

```

30 // server {
```

```
31 // listen 443 ssl http2;
32 // ssl_certificate /etc/letsencrypt/live/myapp.com/fullchain.pem;
33 // ssl_certificate_key /etc/letsencrypt/live/myapp.com/privkey.pem;
34 // location / { proxy_pass http://localhost:3000; }
35 // }
```

■ **Interview Tip:**

Always set `app.set('trust proxy', 1)` when behind a reverse proxy. Without it, `req.headers['x-forwarded-proto']` is ignored and `req.secure` is always false — your redirect middleware never fires and all HTTP traffic goes through unredirected.

How to implement structured request logging in Express?

Answer

Production logging needs to be structured (JSON), queryable, and include the right context.

What every request log should include:

- requestId — link logs across async call chain
- method + url — what was called
- statusCode — outcome (200, 404, 500)
- duration — how long it took in ms
- userId — who made the request (if authenticated)
- ip — client IP address

Log levels by status code:

- 2xx → info level
- 4xx → warn level (client's fault, expected)
- 5xx → error level (our fault, needs attention)

What NOT to log:

- Authorization headers (contain tokens)
- Request body (may contain passwords)
- SSNs, credit card numbers, health data

```
1  const pino = require('pino')
2
3  const logger = pino({
4    level: process.env.LOG_LEVEL || 'info',
5    redact: {
6      paths: ['req.headers.authorization', 'req.body.password', 'req.body.token'],
7      censor: '[REDACTED]' // hide sensitive fields
8    }
9  })
10
11 // Request logger middleware
12 app.use((req, res, next) => {
13   req.startTime = Date.now()
14
15   res.on('finish', () => {
16     const ms = Date.now() - req.startTime
17     const level = res.statusCode >= 500 ? 'error' // server error
18       : res.statusCode >= 400 ? 'warn' // client error
19       : 'info' // success
20
21     logger[level]({
22       requestId: req.id,
23       method: req.method,
24       url: req.originalUrl,
25       status: res.statusCode,
26       ms,
27       ip: req.ip,
```

```

28     userId: req.user?.id || 'anonymous',
29     ua: req.get('user-agent')?.substring(0, 100) // truncate long UAs
30 }, 'http')
31
32 // Alert on very slow requests
33 if (ms > 2000) logger.warn({ requestId: req.id, ms, url: req.originalUrl }, 'SLOW REQUEST')
34 })
35 next()
36 })
37
38 // Skip noisy health check logs
39 const SKIP_LOGGING = new Set(['/health', '/ready', '/metrics'])
40 app.use((req, res, next) => {
41   if (SKIP_LOGGING.has(req.path)) {
42     // Monkey-patch to skip logging for this request
43     return next()
44   }
45   next()
46 })

```

■ Interview Tip:

JSON structured logs are essential in production. Tools like Datadog, Kibana, and CloudWatch Logs Insights let you query: `status:500 AND ms:>1000` — find all errors that were also slow. This is impossible with plain text logs.

Q73 How to implement environment-based config in Express?

■ Answer

Managing configuration across dev/staging/production requires validation at startup.

The problem with missing config:

- Missing JWT_SECRET = tokens accepted without verification
- Missing MONGODB_URI = connecting to wrong database
- These bugs are hard to detect in production

Solution: validate at startup with Zod

- Define all required env vars in schema
- Parse at startup — crash with clear error if missing
- Better to crash at deploy than fail silently in production

Config hierarchy:

1. .env file (local dev — never commit!)
2. Environment variables (staging/production)
3. Defaults in code (non-sensitive only)

```

1 require('dotenv').config() // load .env in development
2 const { z } = require('zod')
3
4 // Validate ALL env vars at startup
5 const envSchema = z.object({
6   NODE_ENV: z.enum(['development', 'test', 'staging', 'production']).default('development'),
7   PORT: z.coerce.number().int().positive().default(3000),
8
9   // Required — crash if missing
10  MONGODB_URI: z.string().url(),
11  REDIS_URL: z.string().url(),
12  JWT_ACCESS_SECRET: z.string().min(32, 'Must be at least 32 chars'),
13  JWT_REFRESH_SECRET: z.string().min(32),
14  CORS_ORIGIN: z.string().url(),
15
16  // Optional
17  S3_BUCKET: z.string().optional(),
18  SENDGRID_KEY: z.string().startsWith('SG.').optional()
19 })
20
21 let config
22 try {
23   config = envSchema.parse(process.env)
24 } catch(err) {
25   console.error('\n■ Invalid environment configuration:')
26   err.errors.forEach(e => console.error(`  ${e.path.join('.')}: ${e.message}`))
27   console.error('\nFix the above variables and restart.\n')
28   process.exit(1) // crash at startup with clear message!
29 }
30
```

```

31 module.exports = Object.freeze(config) // immutable config object

```

```
32
33 // Usage throughout app:
34 const { MONGODB_URI, PORT, JWT_ACCESS_SECRET } = require('./config')
35 mongoose.connect(MONGODB_URI)
36 app.listen(PORT)
37
38 // .env file (in .gitignore!)
39 // NODE_ENV=development
40 // MONGODB_URI=mongodb://localhost:27017/myapp
41 // JWT_ACCESS_SECRET=my-super-secret-key-at-least-32-chars
```

■ **Interview Tip:**

Object.freeze() on the config object prevents accidental mutation. Config should be read-only constants. If someone tries to modify config.PORT in code, it throws in strict mode — catching bugs early.

Q74

How to implement response time tracking in Express?

■ Answer

Tracking response time identifies slow endpoints before users complain.

What to measure:

- Total response time — end-to-end from request to response sent
- Per-route averages — find the consistently slowest endpoints
- P99 latency — worst-case experience for real users
- External call durations — which upstream service is slow

Tools:

- response-time npm package — adds X-Response-Time header
- Custom middleware with process.hrtime.bigint() — nanosecond precision
- Prometheus + Grafana — production metrics dashboards
- APM tools (Datadog, New Relic) — distributed tracing

```

1  const responseTime = require('response-time')
2
3  // Simple: add X-Response-Time header to every response
4  app.use(responseTime())
5  // Response headers: X-Response-Time: 23.456ms
6
7  // Custom: track stats per route for monitoring
8  const routeStats = new Map() // 'GET /users' → { count, totalMs, maxMs }
9
10 app.use((req, res, next) => {
11   const start = process.hrtime.bigint() // nanosecond timer
12
13   res.on('finish', () => {
14     const ms = Number(process.hrtime.bigint() - start) / 1e6 // nanoseconds → ms
15     const route = `${req.method} ${req.route?.path || req.path}`
16
17     const stats = routeStats.get(route) || { count: 0, totalMs: 0, maxMs: 0 }
18     routeStats.set(route, {
19       count: stats.count + 1,
20       totalMs: stats.totalMs + ms,
21       maxMs: Math.max(stats.maxMs, ms)
22     })
23
24     if (ms > 1000) logger.warn({ route, ms }, 'SLOW – over 1 second!')
25   })
26   next()
27 })
28
29 // Expose stats endpoint for monitoring
30 app.get('/internal/perf', authenticate, authorize('admin'), (req, res) => {
31   const metrics = []
32   for (const [route, s] of routeStats) {
33     metrics.push({ route, calls: s.count, avgMs: (s.totalMs/s.count).toFixed(1), maxMs: s.maxMs })

```

```
34     }  
35     res.json(metrics.sort((a, b) => parseFloat(b.avgMs) - parseFloat(a.avgMs)))  
36 }
```

■ Interview Tip:

Track P99 latency, not just averages. If your /products endpoint has avg 50ms but P99 of 3000ms, that means 1% of users wait 3 seconds. With 10,000 requests/day, 100 users per day experience that. Averages hide outlier pain.

What is the full production-ready Express.js setup?

■ Answer

A production Express app combines security, performance, reliability, and observability.

The complete middleware stack in order:

1. helmet — security headers (must be first)
2. cors — allow cross-origin requests
3. compression — gzip responses
4. trust proxy — if behind Nginx/load balancer
5. request ID — every request gets unique ID
6. request logger — log all incoming requests
7. body parser + size limit — parse and limit bodies
8. mongo sanitize — prevent NoSQL injection
9. rate limiter — throttle abusive clients
10. routes — your actual API routes
11. 404 handler — catch unknown routes
12. error handler — centralized error response (LAST)

```
1 require('express-async-errors') // auto-catch async errors (Express 4)
2 const express = require('express')
3 const helmet = require('helmet')
4 const cors = require('cors')
5 const compression = require('compression')
6 const mongoSanitize = require('express-mongo-sanitize')
7 const rateLimit = require('express-rate-limit')
8 const crypto = require('crypto')
9
10 const app = express()
11
12 app.set('trust proxy', 1) // behind Nginx
13 app.disable('x-powered-by') // hide framework
14
15 // Security headers
16 app.use(helmet())
17
18 // CORS
19 app.use(cors({ origin: process.env.CORS_ORIGIN, credentials: true }))
20
21 // Performance
22 app.use(compression())
23
24 // Request ID (must be early)
25 app.use((req, res, next) => {
26   req.id = req.headers['x-request-id'] || crypto.randomUUID()
27   res.setHeader('X-Request-Id', req.id)
28   next()
29 })
30
31 // Body parsing + security
```

```
32 app.use(express.json({ limit: '100kb' })))
```

```

33 app.use(express.urlencoded({ extended: true, limit: '100kb' }))
34 app.use(mongoSanitize()) // NoSQL injection prevention
35
36 // Rate limiting
37 app.use('/api/', rateLimit({ windowMs: 15*60*1000, max: 100 }))
38
39 // Routes
40 app.use('/api/v1', require('./routes'))
41 app.get('/health', (req, res) => res.json({ status: 'ok' }))
42 app.get('/ready', require('./middleware/readinessCheck'))
43
44 // 404 handler
45 app.use((req, res) => res.status(404).json({ error: 'Route not found' }))
46
47 // Error handler — ALWAYS LAST
48 app.use(require('./middleware/errorHandler'))
49
50 module.exports = app // export without listen() for testing

```

■ Interview Tip:

This is the complete production Express boilerplate. Copy-paste this as your starting point for every new project. Every layer has a specific security or performance purpose. Remove layers only intentionally — every line is there for a reason.

Q76 How to implement request validation with Zod in Express?

■ Answer

Zod is the modern standard for runtime validation with full TypeScript type inference.

Why Zod over alternatives:

- Types derived from schema automatically — one source of truth
- Works in both Node.js and browser — share with React forms
- Clean, chainable API with excellent error messages
- Strict by default — unknown fields are stripped

Validate three sources:

- req.body — POST/PUT/PATCH data
- req.params — URL path parameters like :id
- req.query — query string like ?page=1&limit=20

Middleware pattern:

- Create validate(schema) factory function
- Wrap each route with appropriate schema
- Returns 400 with field-level error details on failure

```
1  const { z } = require('zod')
2
3  // Validation middleware factory
4  const validate = (schema) => async (req, res, next) => {
5    try {
6      const parsed = await schema.parseAsync({
7        body: req.body,
8        params: req.params,
9        query: req.query
10     })
11     // Replace with parsed (coerced, trimmed, defaulted) values
12     if (parsed.body) req.body = parsed.body
13     if (parsed.params) req.params = parsed.params
14     if (parsed.query) req.query = parsed.query
15     next()
16   } catch(err) {
17     if (err instanceof z.ZodError) {
18       return res.status(400).json({
19         error: 'Validation failed',
20         details: err.errors.map(e => ({ field: e.path.join('.'), message: e.message }))
21       })
22     }
23     next(err)
24   }
25 }
26
27 // Schemas
28 const mongoId = z.string().regex(/^[0-9a-fA-F]{24}$/, 'Invalid MongoDB ID')
```

```

30 const createProductSchema = z.object({
31   body: z.object({
32     name: z.string().min(2).max(100).trim(),
33     price: z.number().positive().max(1000000),
34     category: z.enum(['electronics', 'clothing', 'food', 'books']),
35     stock: z.number().int().nonnegative().default(0)
36   }),
37   params: z.object({}),
38   query: z.object({})
39 })
40
41 const listSchema = z.object({
42   body: z.object({}),
43   params: z.object({}),
44   query: z.object({
45     page: z.coerce.number().int().positive().default(1),
46     limit: z.coerce.number().int().positive().max(100).default(20),
47     category: z.string().optional()
48   })
49 })
50
51 router.get('/products', validate(listSchema), ctrl.list)
52 router.post('/products', authenticate, validate(createProductSchema), ctrl.create)

```

■ Interview Tip:

Zod's `safeParse()` vs `parse()`: `parse()` throws `ZodError` on failure (use in middleware), `safeParse()` returns `{ success, data, error }` without throwing (use when you need to handle failure in code without try/catch).

Q77 How to implement file download with Express?

■ Answer

Express provides `res.download()` and `res.sendFile()` for serving files to clients.

res.download(path, filename, options):

- Sets Content-Disposition: attachment — browser downloads instead of displaying
- Second param: the filename shown in browser's save dialog
- Handles streaming automatically

res.sendFile(path, options):

- Sets Content-Disposition: inline — browser displays if it can
- Good for serving PDFs, images inline
- Requires absolute path OR root option

Security:

- Always use `path.basename()` on user-provided filenames
- Validate that resolved path is within allowed directory
- Never serve arbitrary filesystem paths

```

1  const path = require('path')
2  const fs = require('fs')
3
4  // Simple file download
5  router.get('/reports/:filename', authenticate, asyncHandler(async (req, res) => {
6    // Security: strip directory traversal
7    const safeName = path.basename(req.params.filename)
8    const filePath = path.join(__dirname, '../reports', safeName)
9
10   // Verify file is within allowed directory
11   const allowed = path.resolve(__dirname, '../reports')
12   if (!filePath.startsWith(allowed)) return res.status(403).json({ error: 'Forbidden' })
13
14   // Check file exists
15   if (!fs.existsSync(filePath)) return res.status(404).json({ error: 'File not found' })
16
17   // Download with custom filename
18   res.download(filePath, `report-${Date.now()}.pdf`, (err) => {
19     if (err && !res.headersSent) res.status(500).json({ error: 'Download failed' })
20   })
21 })
22
23 // Generate and stream file (never saves to disk)
24 router.get('/export/csv', authenticate, asyncHandler(async (req, res) => {
25   const orders = await Order.find({ userId: req.user.id }).lean()
26
27   res.setHeader('Content-Type', 'text/csv')
28   res.setHeader('Content-Disposition', `attachment; filename=orders-${Date.now()}.csv`)
29

```

```

30 // Stream CSV directly to response

```

```

31   res.write('ID,Total,Status,Date\n')
32   for (const o of orders) {
33     res.write(`${o._id},${o.total},${o.status},${o.createdAt.toISOString()}\n`)
34   }
35   res.end()
36 )))
37
38 // Inline PDF (browser displays, not downloads)
39 router.get('/invoice/:id', authenticate, asyncHandler(async (req, res) => {
40   const filePath = path.join(__dirname, '../invoices', `${req.params.id}.pdf`)
41   res.sendFile(filePath, { root: '/' }, (err) => {
42     if (err) res.status(404).json({ error: 'Invoice not found' })
43   })
44 }))

```

■ Interview Tip:

For generated files (CSVs, reports), stream directly to the response — never save to disk first. Temporary files accumulate on disk, disk fills up, server crashes. Streaming is also faster — the client starts receiving data immediately.

Q78 How to implement Express request body size limits and payload protection?

■ Answer

Without size limits, attackers can send GB-sized request bodies to exhaust memory.

Attack: Large payload DoS

- Attacker POSTs 1GB JSON body
- Express buffers entire body in memory
- Server runs out of RAM — crashes

Protection layers:

- `express.json({ limit: '100kb' })` — reject bodies > 100KB
- `express.urlencoded({ limit: '100kb' })`
- Nginx `client_max_body_size` directive

Different limits for different routes:

- API JSON: 100kb (default is sufficient)
- File uploads: 10MB (multer handles this separately)
- Webhook raw body: 1MB

What happens when limit exceeded:

- Express sends 413 Payload Too Large
- Client receives 413 status code automatically

```
1  const express = require('express')
2
3  // Apply body parsers with limits BEFORE routes
4  app.use(express.json({
5    limit: '100kb', // 100KB for JSON APIs
6    strict: true    // only accept objects and arrays (not primitives)
7  }))
8
9  app.use(express.urlencoded({
10    extended: true,
11    limit: '100kb' // 100KB for form submissions
12  }))
13
14 // Different limits for specific routes
15 app.use('/api/webhooks', express.raw({
16   type: 'application/json',
17   limit: '1mb' // webhooks may be larger
18 }))
19
20 // Multer handles file uploads with its own limits
21 const upload = multer({ limits: { fileSize: 10 * 1024 * 1024 } }) // 10MB files
22
23 // Handle 413 errors from body parser
24 app.use((err, req, res, next) => {
25   if (err.status === 413 || err.type === 'entity.too.large') {
26     return res.status(413).json({
```

```

27     error: 'Payload too large',
28     maxSize: '100KB',
29     hint: 'Split your request into smaller batches'
30   })
31 }
32 next(err)
33 })
34
35 // Also protect in Nginx (before Express):
36 // nginx.conf:
37 // client_max_body_size 10m; // 10MB max
38 // For file uploads, set this higher in upload locations only
39
40 // Validate content-type to prevent unexpected parsing
41 app.use('/api/', (req, res, next) => {
42   if (['POST', 'PUT', 'PATCH'].includes(req.method)) {
43     const ct = req.headers['content-type'] || ''
44     if (!ct.includes('application/json') && !ct.includes('multipart/form-data')) {
45       return res.status(415).json({ error: 'Unsupported Media Type' })
46     }
47   }
48   next()
49 })

```

■ Interview Tip:

Set body size limits to the minimum your application actually needs. If your API only receives small JSON objects, set limit to 10kb not 100kb. The tighter the limit, the less damage a DoS attack can do even if it gets through rate limiting.

Q79 How to implement CORS for multiple environments in Express?

■ Answer

CORS configuration needs to be environment-aware — different origins in dev vs production.

Environment-specific origins:

- Development: localhost:3000, localhost:5173 (Vite), localhost:4200 (Angular)
- Staging: staging.app.com
- Production: app.com, admin.app.com

Dynamic origin validation:

- Check incoming origin against allowlist
- Allow requests without Origin (curl, Postman, server-to-server)
- Reject unrecognized origins with CORS error

credentials: true requirement:

- Must use specific origins (not *) when credentials:true
- Required for cookie-based auth across origins

```

1  const cors = require('cors')
2
3  // Build origin allowlist from environment
4  function buildAllowedOrigins() {
5      const origins = new Set()
6
7      if (process.env.CORS_ORIGINS) {
8          process.env.CORS_ORIGINS.split(',').forEach(o => origins.add(o.trim()))
9      }
10
11     if (process.env.NODE_ENV === 'development') {
12         origins.add('http://localhost:3000') // React CRA
13         origins.add('http://localhost:5173') // Vite
14         origins.add('http://localhost:4200') // Angular
15         origins.add('http://localhost:8080') // Vue CLI
16     }
17
18     return origins
19 }
20
21 const allowedOrigins = buildAllowedOrigins()
22 console.log('CORS allowed origins:', [...allowedOrigins])
23
24 app.use(cors({
25     origin: (origin, callback) => {
26         // Allow requests without origin (Postman, curl, server-to-server)
27         if (!origin) return callback(null, true)
28
29         if (allowedOrigins.has(origin)) {
30             callback(null, true) // allowed
31
32         } else {

```

```

32     logger.warn({ origin }, 'CORS blocked – unauthorized origin')
33     callback(new Error(`CORS: ${origin} is not allowed`))
34   }
35 },
36 credentials: true, // allow cookies
37 methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
38 allowedHeaders: ['Content-Type', 'Authorization', 'X-Request-Id'],
39 exposedHeaders: ['X-Request-Id', 'X-RateLimit-Remaining'], // expose to browser JS
40 maxAge: 86400 // preflight cached for 24 hours
41 })))
42
43 // Handle preflight for all routes
44 app.options('*', cors()) // must explicitly handle OPTIONS

```

■ Interview Tip:

Set CORS_ORIGINS as a comma-separated env var: CORS_ORIGINS=https://app.com,https://admin.app.com. This makes CORS config visible in your deployment config (Kubernetes ConfigMap, Docker Compose) rather than buried in code.

Q80 How to structure a large Express.js application?

■ Answer

As Express apps grow, structure becomes critical for maintainability and testability.

Recommended folder structure:

src/
routes/ — URL routing only, no business logic
controllers/ — req/res handling, calls services
services/ — business logic, calls repositories
repositories/ — database queries only
models/ — Mongoose schemas and models
middleware/ — auth, validate, rateLimit, error handler
config/ — env validation, db connection, redis
utils/ — helpers: asyncHandler, logger, errors
types/ — TypeScript type declarations

Data flow:

Request → Router → Middleware → Controller → Service → Repository → DB

File naming convention:

- users.routes.js, users.controller.js, users.service.js

```
1 // src/app.js - just wires up middleware and routes
2 const express = require('express')
3 const { setupMiddleware } = require('./middleware')
4 const routes = require('./routes')
5 const { errorHandler } = require('./middleware/errorHandler')
6
7 const app = express()
8 setupMiddleware(app) // helmet, cors, compression, body parsing
9 app.use('/api/v1', routes) // all routes
10 app.use(errorHandler) // must be last
11 module.exports = app
12
13 // src/routes/index.js - barrel file
14 const router = require('express').Router()
15 router.use('/auth', require('./auth.routes'))
16 router.use('/users', require('./users.routes'))
17 router.use('/products', require('./products.routes'))
18 module.exports = router
19
20 // src/routes/users.routes.js - routing only
21 const router = require('express').Router()
22 const ctrl = require('../controllers/users.controller')
23 const { authenticate, authorize } = require('../middleware/auth')
24 const { validate } = require('../middleware/validate')
25 const { createUserSchema } = require('../schemas/user.schema')
26 router.get('/', authenticate, authorize('admin'), ctrl.list)
27 router.post('/', validate(createUserSchema), ctrl.create)
28 module.exports = router
```

```

29
30 // src/controllers/users.controller.js – thin HTTP layer
31 const userService = require('../services/users.service')
32 const { asyncHandler } = require('../utils/asyncHandler')
33 exports.list = asyncHandler(async (req, res) => {
34   const users = await userService.findAll(req.query) // delegate to service
35   res.json(users)
36 })
37
38 // src/services/users.service.js – business logic
39 const userRepo = require('../repositories/users.repository')
40 exports.findAll = async (filters) => {
41   return userRepo.findActive(filters) // delegate to repository
42 }
43
44 // src/repositories/users.repository.js – DB queries only
45 const User = require('../models/User')
46 exports.findActive = (filters) => User.find({ active: true, ...filters }).lean()

```

■ Interview Tip:

Keep each layer single-responsibility: routes know about URLs and HTTP methods, controllers know about req/res, services know about business rules, repositories know about database queries. This lets you test each layer independently with mocks.

Express.js 80 Q — Complete! ■

■ All 80 questions with detailed answers + code!

Next: MongoDB (120 Q) → Auth & Security (60 Q) → System Design (90 Q)

■ SAVE — your Express.js bible

■■ REPOST — help your dev community

■ COMMENT 'JS950' — get all 950 Q PDF

■ FOLLOW Kaushal Singh — daily prep

Kaushal Singh

Full-Stack Dev | MERN + React Native | Next.js

1M+ Impressions | React · React Native · Express · Next · Node · MongoDB · Interview Prep daily

Section 4/10 done | Next: MongoDB 120 Q | Comment 'JS950'